MODUL 05 · INPUT & FORM

Interaktivitas, State Dasar & Validasi Formulir

Membangun antarmuka interaktif dengan tombol Material 3 dan gestur, mengelola input teks dengan TextEditingController dan FocusNode, memvalidasi formulir dengan GlobalKey<FormState>, memakai input pilihan (Checkbox, Switch, RadioGroup, Slider, Dropdown, DatePicker), serta memberi umpan balik lewat SnackBar, AlertDialog, dan BottomSheet.

Durasi Pertemuan 5 (150 Menit) + Belajar Mandiri

Prasyarat Modul 04: Sistem Layouting

Capaian Pembelajaran

- Memilih varian tombol Material 3 (FilledButton, ElevatedButton, OutlinedButton, TextButton, IconButton, FAB) sesuai hierarki aksi dan menonaktifkannya dengan onPressed null.
- Membuat widget kustom dapat disentuh dengan GestureDetector dan InkWell.
- Mengatur tampilan TextField dengan InputDecoration serta mengelola TextEditingController dan FocusNode beserta dispose-nya.
- Menerapkan validasi form lengkap (email, password, konfirmasi password) dengan Form, TextFormField, dan GlobalKey<FormState>.
- Menggunakan Checkbox, Switch, RadioGroup, Slider, DropdownButtonFormField, DatePicker, dan TimePicker.
- Memberi umpan balik kepada pengguna dengan SnackBar, AlertDialog, dan BottomSheet.
- Membangun formulir pendaftaran mahasiswa lengkap beserta halaman hasilnya.

Daftar Isi Modul

1. Ragam Tombol Material 3
2. GestureDetector & InkWell
3. TextField & InputDecoration
4. TextEditingController: Baca, Ubah, Kosongkan
5. Jenis Keyboard, Password & FocusNode
6. Form, TextFormField & Validasi
7. Checkbox, Switch, Radio & Slider
8. Dropdown, DatePicker & TimePicker
9. Umpan Balik: SnackBar, AlertDialog & BottomSheet
10. Studi Kasus: Formulir Pendaftaran Mahasiswa
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Ragam Tombol Material 3

Tombol adalah cara paling dasar pengguna "berbicara" dengan aplikasi. Material Design 3 (M3) menyediakan beberapa jenis tombol dengan **hierarki** yang jelas: aksi paling penting dibuat paling menonjol, aksi pendukung dibuat lebih kalem.

Konsep: memilih tombol sesuai tingkat kepentingan

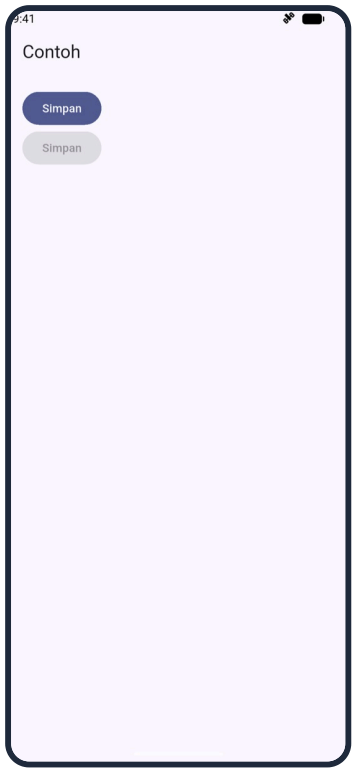
Widget	Kapan dipakai	Contoh di aplikasi kampus
`FilledButton`	Aksi utama, paling menonjol. Idealnya satu per layar.	Simpan KRS, Kirim, Masuk
`FilledButton.tonal`	Aksi penting tapi bukan yang utama.	Simpan sebagai draft
`ElevatedButton`	Tombol berbayang, dipakai jika latar belakang ramai sehingga tombol perlu "terangkat".	Lihat KHS di atas gambar
`OutlinedButton`	Aksi sekunder, sering berpasangan dengan FilledButton.	Batal, Kembali
`TextButton`	Aksi prioritas rendah, tampil seperti tautan.	Lupa password?, Lihat semua
`IconButton`	Aksi berupa ikon saja, di AppBar, kartu, atau baris daftar.	Hapus, Favorit, Bagikan
`FloatingActionButton` (FAB)	Aksi utama sebuah layar yang "melayang" di pojok kanan bawah.	Tambah catatan, Isi KRS

Contoh kecil: tombol aktif dan tombol nonaktif

Semua tombol di atas punya dua parameter utama: `onPressed` (fungsi yang dijalankan saat ditekan) dan `child` (isi tombol, biasanya `Text`). Aturan pentingnya: **jika `onPressed` bernilai `null`, tombol otomatis menjadi nonaktif (disabled)**, warnanya memudar dan tidak bisa ditekan.

```
// Tombol AKTIF: onPressed berisi fungsi
FilledButton(
  onPressed: () {
    debugPrint('Tombol Simpan ditekan');
  },
  child: const Text('Simpan'),
),
```

```dart
// Tombol NONAKTIF (disabled): onPressed bernilai null
const FilledButton(
  onPressed: null,
  child: Text('Simpan'),
),
```



Atas: tombol aktif. Bawah: tombol dengan onPressed null (disabled)

Contoh lengkap: galeri tombol Material 3

Program berikut menampilkan semua jenis tombol, menghitung jumlah klik, dan punya satu tombol untuk menonaktifkan semua tombol lain sekaligus. Salin ke `lib/main.dart` lalu jalankan.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const GaleriTombolScreen(),
    );
  }
```

```dart
}

class GaleriTombolScreen extends StatefulWidget {
  const GaleriTombolScreen({super.key});

  @override
  State<GaleriTombolScreen> createState() => _GaleriTombolScreenState();
}

class _GaleriTombolScreenState extends State<GaleriTombolScreen> {
  int _jumlahKlik = 0;
  bool _tombolAktif = true;
  String _aksiTerakhir = '-';

  void _catat(String aksi) {
    setState(() {
      _jumlahKlik++;
      _aksiTerakhir = aksi;
    });
  }

  @override
  Widget build(BuildContext context) {
    // Jika _tombolAktif false, semua callback menjadi null -> tombol disabled
    final VoidCallback? aksiSimpan = _tombolAktif ? () => _catat('Simpan') : null;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Galeri Tombol M3'),
        actions: [
          IconButton(
            tooltip: 'Reset',
            icon: const Icon(Icons.refresh),
            onPressed: () => setState(() {
              _jumlahKlik = 0;
              _aksiTerakhir = '-';
            }),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          Text('Jumlah klik: $_jumlahKlik', style: Theme.of(context).textTheme.headlineSmall),
          Text('Aksi terakhir: $_aksiTerakhir'),
          const Divider(height: 32),

          // 1. Prioritas tertinggi
          FilledButton(onPressed: aksiSimpan, child: const Text('FilledButton - Simpan')),
          const SizedBox(height: 8),
          FilledButton.tonal(
            onPressed: _tombolAktif ? () => _catat('Tonal') : null,
            child: const Text('FilledButton.tonal - Draft'),
          ),
          const SizedBox(height: 8),

          // 2. Tombol dengan bayangan (elevasi)
          ElevatedButton(
            onPressed: _tombolAktif ? () => _catat('Elevated') : null,
```

```
    child: const Text('ElevatedButton - Lihat KHS'),
  ),
  const SizedBox(height: 8),

  // 3. Aksi sekunder
  OutlinedButton(
    onPressed: _tombolAktif ? () => _catat('Batal') : null,
    child: const Text('OutlinedButton - Batal'),
  ),
  const SizedBox(height: 8),

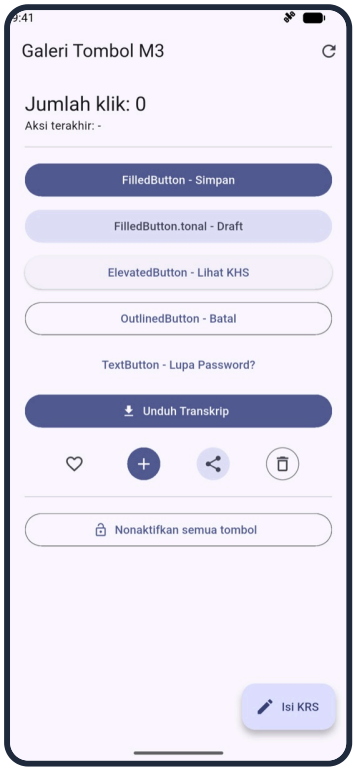
  // 4. Aksi prioritas rendah
  TextButton(
    onPressed: _tombolAktif ? () => _catat('Lupa Password') : null,
    child: const Text('TextButton - Lupa Password?'),
  ),
  const SizedBox(height: 8),

  // 5. Versi .icon: ikon + label
  FilledButton.icon(
    onPressed: _tombolAktif ? () => _catat('Unduh') : null,
    icon: const Icon(Icons.download),
    label: const Text('Unduh Transkrip'),
  ),
  const SizedBox(height: 16),

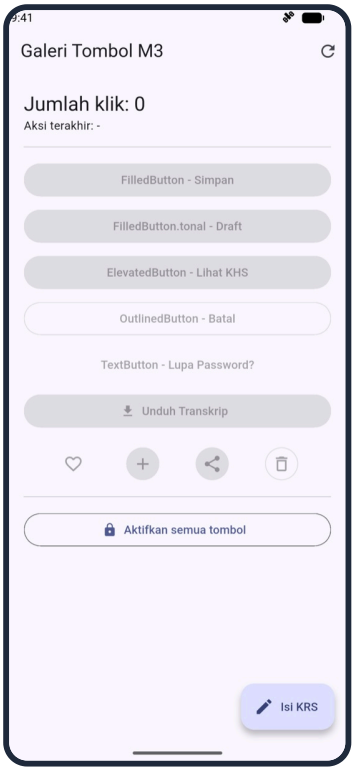
  // 6. IconButton dan variannya
  Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      IconButton(
        tooltip: 'Favorit',
        onPressed: _tombolAktif ? () => _catat('Favorit') : null,
        icon: const Icon(Icons.favorite_border),
      ),
      IconButton.filled(
        tooltip: 'Tambah',
        onPressed: _tombolAktif ? () => _catat('Tambah') : null,
        icon: const Icon(Icons.add),
      ),
      IconButton.filledTonal(
        tooltip: 'Bagikan',
        onPressed: _tombolAktif ? () => _catat('Bagikan') : null,
        icon: const Icon(Icons.share),
      ),
      IconButton.outlined(
        tooltip: 'Hapus',
        onPressed: _tombolAktif ? () => _catat('Hapus') : null,
        icon: const Icon(Icons.delete_outline),
      ),
    ],
  ),
  const Divider(height: 32),

  // Saklar untuk mengaktifkan / menonaktifkan semua tombol
  OutlinedButton.icon(
    onPressed: () => setState(() => _tombolAktif = !_tombolAktif),
    icon: Icon(_tombolAktif ? Icons.lock_open : Icons.lock),
    label: Text(_tombolAktif ? 'Nonaktifkan semua tombol' : 'Aktifkan semua tombol'),
```

```
          ),
          const SizedBox(height: 80), // ruang agar FAB tidak menutupi tombol terakhir
        ],
      ),
      // 7. FloatingActionButton: aksi utama layar
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _catat('FAB'),
        icon: const Icon(Icons.edit),
        label: const Text('Isi KRS'),
      ),
    );
  }
}
```



Galeri tombol saat semua tombol aktif



Setelah menekan "Nonaktifkan semua tombol"

Penjelasan kode:

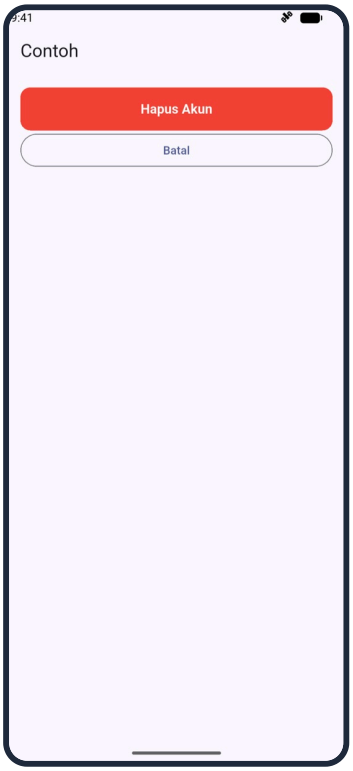
- `final VoidCallback? aksiSimpan = _tombolAktif ? () => _catat('Simpan') : null;` adalah pola paling umum untuk mengaktifkan atau menonaktifkan tombol berdasarkan kondisi. `VoidCallback?` berarti "fungsi tanpa parameter, boleh null".
- `FilledButton.icon`, `OutlinedButton.icon`, dan sejenisnya menerima `icon` dan `label` sebagai pengganti `child`.
- `IconButton.filled`, `.filledTonal`, dan `.outlined` adalah varian IconButton khas M3. Selalu isi `tooltip` agar pembaca layar (aksesibilitas) tahu fungsi ikon tersebut.
- `FloatingActionButton.extended` adalah FAB dengan ikon dan teks. Karena FAB menutupi sebagian konten, kita beri `SizedBox(height: 80)` di akhir daftar.

Mengubah gaya tombol

Setiap jenis tombol punya fungsi statis `styleFrom()` untuk mengubah warna, ukuran, dan bentuk. Untuk tombol selebar layar, cara paling mudah adalah `minimumSize: Size.fromHeight(...)` atau membungkus tombol dengan `SizedBox(width: double.infinity)`.

```
// Mengubah gaya satu tombol dengan styleFrom
FilledButton(
  style: FilledButton.styleFrom(
    backgroundColor: Colors.red,          // warna latar
    foregroundColor: Colors.white,        // warna teks & ikon
    minimumSize: const Size.fromHeight(52), // tinggi 52, lebar penuh
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
  ),
  onPressed: () {},
  child: const Text('Hapus Akun'),
),

// Tombol selebar layar tanpa styleFrom: bungkus dengan SizedBox
SizedBox(
  width: double.infinity,
  child: OutlinedButton(onPressed: () {}, child: const Text('Batal')),
),
```



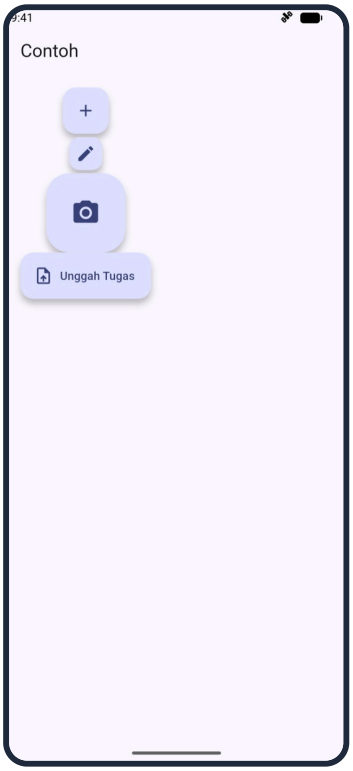
Tombol merah selebar layar dengan styleFrom dan OutlinedButton selebar layar

Ragam FloatingActionButton

```dart
// FAB standar (ikon saja)
FloatingActionButton(
  onPressed: () {},
  tooltip: 'Tambah catatan',
  child: const Icon(Icons.add),
),

// FAB kecil dan besar
FloatingActionButton.small(onPressed: () {}, child: const Icon(Icons.edit)),
FloatingActionButton.large(onPressed: () {}, child: const Icon(Icons.camera_alt)),

// FAB dengan label teks
FloatingActionButton.extended(
  onPressed: () {},
  icon: const Icon(Icons.upload_file),
  label: const Text('Unggah Tugas'),
),
```



FAB standar, small, large, dan extended

> ℹ️ **FAB biasanya hanya satu**
>
> Contoh di atas sengaja menampilkan beberapa FAB untuk perbandingan. Dalam aplikasi nyata, satu layar sebaiknya hanya punya **satu** FAB yang dipasang di parameter `floatingActionButton` milik `Scaffold`. Jika terpaksa memakai beberapa FAB di satu layar, beri masing-masing `heroTag` yang berbeda agar tidak terjadi error saat pindah halaman.

⚠️ **Kesalahan umum**

- **Menulis `onPressed: _simpan()` (dengan tanda kurung).** Ini MEMANGGIL fungsi saat build, bukan saat ditekan. Tulis `onPressed: _simpan` atau `onPressed: () => _simpan()`.
- **Bingung kenapa tombol abu-abu dan tidak bisa ditekan.** Periksa apakah `onPressed` bernilai `null`.
- **Memakai banyak `FilledButton` dalam satu layar.** Pengguna jadi bingung mana aksi utama. Pakai Outlined atau Text untuk aksi pendukung.
- **Lupa `tooltip` pada `IconButton`.** Ikon saja sering tidak jelas maknanya.

2. GestureDetector & InkWell

Tidak semua yang bisa ditekan harus berbentuk tombol. Kartu mata kuliah, foto profil, atau kotak menu juga bisa dibuat responsif terhadap sentuhan menggunakan `GestureDetector` atau `InkWell`.

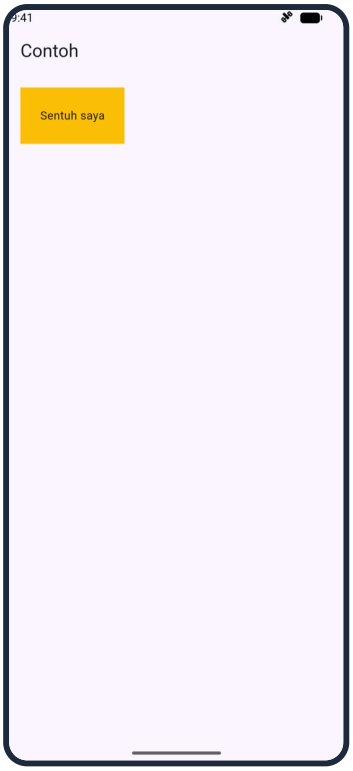
Konsep

	`GestureDetector`	`InkWell`
Efek visual saat ditekan	Tidak ada	Riak air (*ripple*) khas Material
Jenis gestur	Sangat lengkap: tap, double tap, long press, drag, pan, scale	Tap, double tap, long press, hover
Syarat	Tidak ada	Harus ada widget `Material` (atau `Card`, `Scaffold`) di atasnya
Cocok untuk	Gestur khusus (geser, cubit), area tanpa efek	Kartu atau baris yang bisa diklik

Contoh kecil: GestureDetector

Bungkus widget apa pun dengan `GestureDetector` lalu isi callback gestur yang ingin ditangkap.

```
GestureDetector(
  onTap: () => debugPrint('Tap sekali'),
  onDoubleTap: () => debugPrint('Tap dua kali'),
  onLongPress: () => debugPrint('Tekan lama'),
  child: Container(
    padding: const EdgeInsets.all(24),
    color: Colors.amber,
    child: const Text('Sentuh saya'),
  ),
),
```

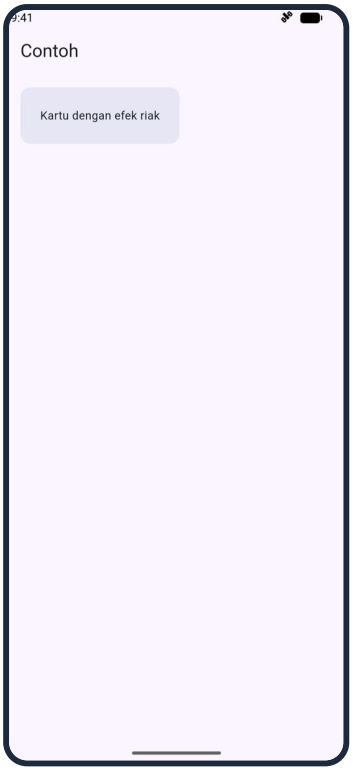


Kotak kuning yang bisa disentuh. Tidak ada efek visual apa pun saat ditekan

Contoh kecil: InkWell

InkWell menggambar efek riak di atas widget `Material` terdekat. Karena itu warna latar sebaiknya diberikan ke `Material`, **bukan** ke `Container` di dalam InkWell (Container berwarna akan menutupi riaknya).

```
// Material memberi "kanvas" tempat efek riak digambar
Material(
  color: Colors.indigo.shade50,
  borderRadius: BorderRadius.circular(12),
  child: InkWell(
    borderRadius: BorderRadius.circular(12), // riak ikut melengkung
    onTap: () => debugPrint('Kartu ditekan'),
    child: const Padding(
      padding: EdgeInsets.all(24),
      child: Text('Kartu dengan efek riak'),
    ),
  ),
),
```



Kartu dengan efek riak saat ditekan

Contoh lengkap: berbagai gestur dalam satu layar

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const GestureDemoScreen(),
    );
  }
}

class GestureDemoScreen extends StatefulWidget {
  const GestureDemoScreen({super.key});

  @override
  State<GestureDemoScreen> createState() => _GestureDemoScreenState();
}

class _GestureDemoScreenState extends State<GestureDemoScreen> {
  String _log = 'Belum ada gestur';
```

```dart
  bool _disukai = false;
  double _geserX = 0;

  void _tulisLog(String pesan) => setState(() => _log = pesan);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('GestureDetector & InkWell')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Card(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Text('Log: $_log', style: const TextStyle(fontSize: 16)),
            ),
          ),
          const SizedBox(height: 16),

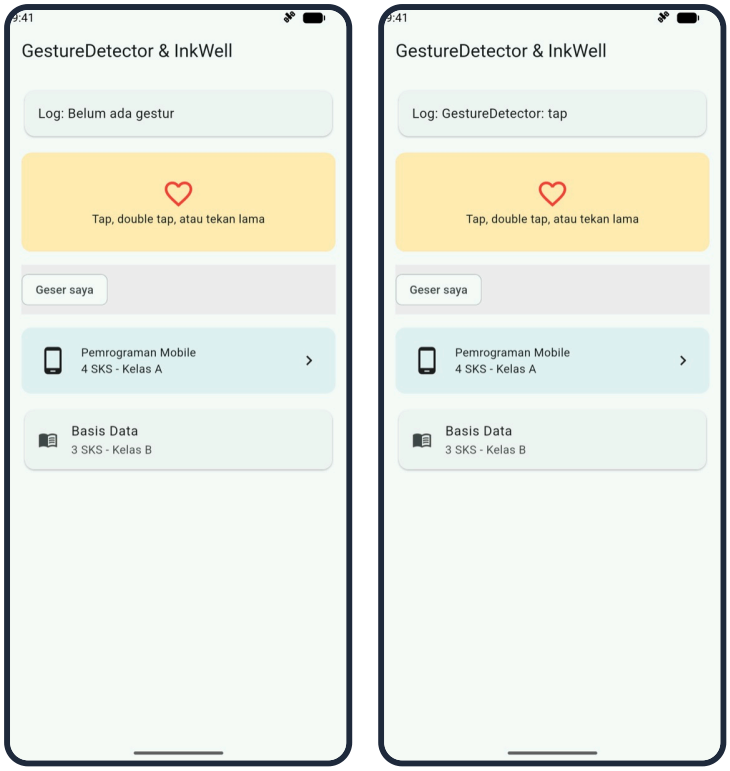
          // 1. GestureDetector: banyak jenis gestur, TANPA efek visual
          GestureDetector(
            onTap: () => _tulisLog('GestureDetector: tap'),
            onDoubleTap: () {
              setState(() => _disukai = !_disukai);
              _tulisLog('GestureDetector: double tap -> suka = $_disukai');
            },
            onLongPress: () => _tulisLog('GestureDetector: long press'),
            child: Container(
              height: 120,
              decoration: BoxDecoration(
                color: Colors.amber.shade100,
                borderRadius: BorderRadius.circular(12),
              ),
              alignment: Alignment.center,
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Icon(
                    _disukai ? Icons.favorite : Icons.favorite_border,
                    color: Colors.red,
                    size: 40,
                  ),
                  const Text('Tap, double tap, atau tekan lama'),
                ],
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 2. GestureDetector untuk drag horizontal
          GestureDetector(
            onHorizontalDragUpdate: (details) {
              setState(() => _geserX = (_geserX + details.delta.dx).clamp(0, 200));
            },
            onHorizontalDragEnd: (_) => _tulisLog('Geser selesai di x =
${_geserX.toStringAsFixed(0)}'),
            child: Container(
              height: 60,
```

```
              color: Colors.grey.shade200,
              alignment: Alignment.centerLeft,
              child: Transform.translate(
                offset: Offset(_geserX, 0),
                child: const Chip(label: Text('Geser saya')),
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 3. InkWell: efek riak Material saat ditekan
          Material(
            color: Colors.teal.shade50,
            borderRadius: BorderRadius.circular(12),
            child: InkWell(
              borderRadius: BorderRadius.circular(12),
              onTap: () => _tulisLog('InkWell: kartu Pemrograman Mobile ditekan'),
              onLongPress: () => _tulisLog('InkWell: long press'),
              child: const Padding(
                padding: EdgeInsets.all(20),
                child: Row(
                  children: [
                    Icon(Icons.phone_android, size: 36),
                    SizedBox(width: 16),
                    Expanded(
                      child: Text('Pemrograman Mobile\n4 SKS - Kelas A'),
                    ),
                    Icon(Icons.chevron_right),
                  ],
                ),
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 4. Cara paling ringkas: Card + InkWell (Card sudah berupa Material)
          Card(
            clipBehavior: Clip.antiAlias, // agar riak tidak keluar dari sudut kartu
            child: InkWell(
              onTap: () => _tulisLog('InkWell di dalam Card ditekan'),
              child: const ListTile(
                leading: Icon(Icons.menu_book),
                title: Text('Basis Data'),
                subtitle: Text('3 SKS - Kelas B'),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



Tampilan awal

Setelah kotak kuning diketuk: log berubah

Penjelasan kode:

- **Kotak kuning** memakai `GestureDetector` dengan tiga callback. Double tap membalik nilai `_disukai` sehingga ikon hati berganti.
- **Chip "Geser saya"** memakai `onHorizontalDragUpdate`. `details.delta.dx` adalah jarak geser jari sejak frame sebelumnya; nilainya dijumlahkan lalu dibatasi dengan `clamp(0, 200)`.
- **Kartu Pemrograman Mobile** memakai pola `Material` + `InkWell`. Nilai `borderRadius` diberikan ke keduanya agar riak ikut melengkung.
- **Kartu Basis Data** memakai `Card`, yang sudah berupa Material. `clipBehavior: Clip.antiAlias` memotong riak agar tidak keluar dari sudut kartu.

> 💡 **Tips**
>
> Untuk baris daftar, `ListTile` sudah punya `onTap` dan efek riak sendiri, jadi tidak perlu dibungkus InkWell. Untuk tombol, gunakan widget tombol bawaan. InkWell paling berguna untuk kartu atau tata letak kustom.

⚠️ **Kesalahan umum**

- **Riak InkWell tidak terlihat.** Biasanya karena tidak ada `Material` di atasnya, atau anak InkWell adalah `Container` berwarna yang menutupi riak.
- **Area kosong di GestureDetector tidak bisa ditekan.** Secara bawaan hanya bagian yang "terlihat" yang menerima sentuhan. Tambahkan `behavior: HitTestBehavior.opaque` jika seluruh area harus responsif.
- **Memasang GestureDetector di atas tombol.** Tombol sudah menangani sentuhannya sendiri, sehingga kedua penangan bisa saling berebut.

3. TextField & InputDecoration

`TextField` adalah kotak tempat pengguna mengetik teks. Tampilannya (label, ikon, garis tepi, pesan error) diatur oleh satu objek bernama `InputDecoration`.

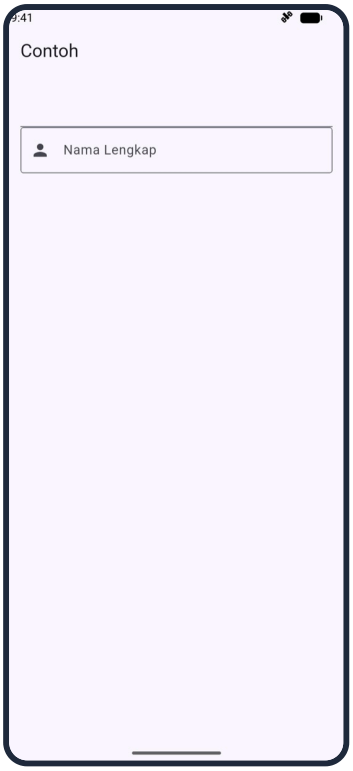
Konsep

- `TextField` mengurus *perilaku*: menerima ketikan, menampilkan kursor, membuka keyboard.
- `InputDecoration` mengurus *tampilan*: `labelText`, `hintText`, `prefixIcon`, `suffixIcon`, `helperText`, `errorText`, `border`, dan lain-lain.
- Untuk membaca isi teks, cara paling sederhana adalah `onChanged`: fungsi yang dipanggil setiap kali isi berubah. Cara yang lebih lengkap (controller) dibahas di section berikutnya.

Contoh kecil

```
// Versi paling sederhana
const TextField(),

// Dengan label, petunjuk, dan ikon
TextField(
  decoration: const InputDecoration(
    labelText: 'Nama Lengkap',        // label yang naik ke atas saat fokus
    hintText: 'Contoh: Nadia Putri',  // teks samar saat kosong
    prefixIcon: Icon(Icons.person),   // ikon di kiri
    border: OutlineInputBorder(),     // kotak bergaris
  ),
  onChanged: (teks) => debugPrint('Diketik: $teks'),
),
```



TextField polos (atas) dan TextField dengan label, hint, ikon, serta kotak garis

Properti InputDecoration	Fungsi
`labelText`	Label di dalam kotak, naik ke atas saat field fokus atau berisi
`hintText`	Contoh isian yang tampil samar saat kosong
`helperText`	Teks bantuan kecil di bawah field
`errorText`	Pesan error merah di bawah field (garis ikut merah)
`prefixIcon` / `suffixIcon`	Ikon di kiri / kanan
`prefixText` / `suffixText`	Teks tetap di kiri / kanan, misalnya "Rp" atau "kg"
`border`	`UnderlineInputBorder()` (bawaan) atau `OutlineInputBorder()`
`filled` + `fillColor`	Memberi warna latar pada field

Contoh lengkap: katalog gaya TextField

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KatalogTextFieldScreen(),
    );
  }
}

class KatalogTextFieldScreen extends StatefulWidget {
  const KatalogTextFieldScreen({super.key});

  @override
  State<KatalogTextFieldScreen> createState() => _KatalogTextFieldScreenState();
}

class _KatalogTextFieldScreenState extends State<KatalogTextFieldScreen> {
  String _nama = '';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Katalog TextField')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Gaya garis bawah (bawaan)
          const TextField(
            decoration: InputDecoration(labelText: '1. Gaya bawaan (garis bawah)'),
          ),
          const SizedBox(height: 16),

          // 2. Gaya kotak + onChanged untuk membaca teks secara langsung
          TextField(
            decoration: const InputDecoration(
              labelText: '2. Nama Panggilan',
              hintText: 'Ketik nama panggilanmu',
              border: OutlineInputBorder(),
            ),
            onChanged: (value) => setState(() => _nama = value),
          ),
          Padding(
            padding: const EdgeInsets.only(top: 4, left: 12),
```

```
        child: Text('Halo, ${_nama.isEmpty ? '...' : _nama}!'),
      ),
      const SizedBox(height: 16),

      // 3. Latar terisi (filled) + ikon depan dan belakang
      const TextField(
        decoration: InputDecoration(
          labelText: '3. Email Kampus',
          prefixIcon: Icon(Icons.email_outlined),
          suffixIcon: Icon(Icons.check_circle_outline),
          filled: true,
        ),
      ),
      const SizedBox(height: 16),

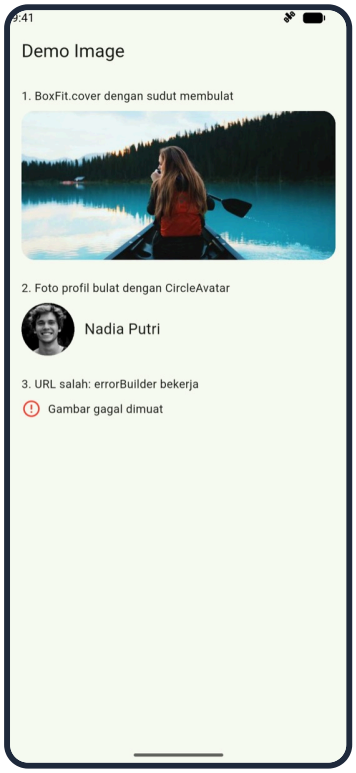
      // 4. Teks bantuan + batas karakter (counter otomatis muncul)
      const TextField(
        maxLength: 12,
        decoration: InputDecoration(
          labelText: '4. NIM',
          helperText: 'Lihat di Kartu Tanda Mahasiswa',
          prefixText: 'H1D',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

      // 5. Multi-baris untuk teks panjang
      const TextField(
        minLines: 3,
        maxLines: 5,
        decoration: InputDecoration(
          labelText: '5. Alamat Kos',
          alignLabelWithHint: true,
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

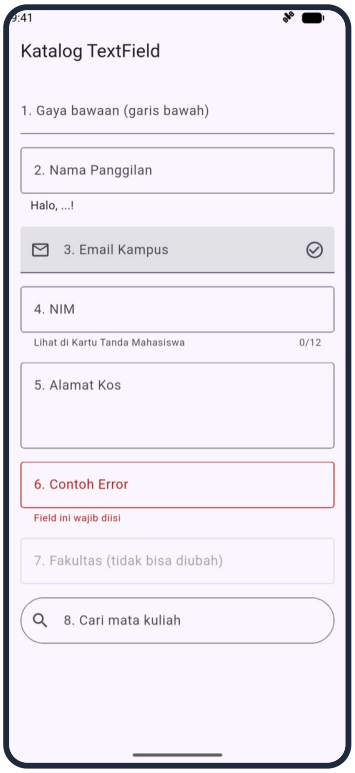
      // 6. Tampilan error manual (tanpa Form)
      const TextField(
        decoration: InputDecoration(
          labelText: '6. Contoh Error',
          errorText: 'Field ini wajib diisi',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

      // 7. Nonaktif dan hanya-baca
      const TextField(
        enabled: false,
        decoration: InputDecoration(
          labelText: '7. Fakultas (tidak bisa diubah)',
          hintText: 'Fakultas Teknik',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),
```

```
          // 8. Sudut membulat + warna garis saat fokus
          TextField(
            decoration: InputDecoration(
              labelText: '8. Cari mata kuliah',
              prefixIcon: const Icon(Icons.search),
              border: OutlineInputBorder(borderRadius: BorderRadius.circular(30)),
              focusedBorder: OutlineInputBorder(
                borderRadius: BorderRadius.circular(30),
                borderSide: const BorderSide(color: Colors.orange, width: 2),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



Bagian atas katalog TextField



Setelah mengetik "Budi" di field nomor 2

Penjelasan kode:

- **Field 2** memakai `onChanged` untuk menyimpan teks ke `_nama` lalu memanggil `setState`, sehingga sapaan di bawahnya ikut berubah setiap huruf diketik.
- **Field 4** memakai `maxLength: 12`. Flutter otomatis menampilkan penghitung "0/12" dan menolak huruf ke-13.
- **Field 5** memakai `minLines` dan `maxLines` sehingga kotak setinggi 3 baris dan bisa membesar sampai 5 baris. `alignLabelWithHint: true` membuat label sejajar di atas, bukan di tengah.

- **Field 6** menampilkan error secara manual dengan `errorText`. Pada form sungguhan, error lebih mudah diatur dengan `Form` dan `validator` (section 6).
- **Field 8** membedakan garis saat normal (`border`) dan saat fokus (`focusedBorder`).

> 💡 **Tema global**
>
> Agar semua TextField di aplikasi otomatis memakai kotak garis, atur sekali di tema: `ThemeData(inputDecorationTheme: const InputDecorationThemeData(border: OutlineInputBorder()))`. Dengan begitu kita tidak perlu menulis `border:` di setiap field.

> ⚠️ **Kesalahan umum**
>
> - **Error "InputDecorator ... unbounded width".** Terjadi saat TextField diletakkan langsung di dalam `Row`. Bungkus dengan `Expanded` atau beri lebar dengan `SizedBox`.
> - **Error "No Material widget found".** TextField harus berada di bawah `Scaffold` atau `Material`.
> - **Keyboard menutupi field di bagian bawah layar.** Pakai `ListView` atau `SingleChildScrollView` sebagai pembungkus agar halaman bisa digulir.

4. TextEditingController: Baca, Ubah, Kosongkan

`onChanged` cukup untuk membaca teks. Namun sering kita juga perlu **mengubah** isi field dari kode, **mengosongkan** field setelah tombol Simpan ditekan, atau **memberi nilai awal**. Untuk itu Flutter menyediakan `TextEditingController`.

Konsep: tiga langkah wajib

1. **Buat** controller sebagai field di dalam `State` (bukan di dalam `build`).
2. **Hubungkan** ke TextField lewat parameter `controller:`.
3. **Buang** (`dispose()`) controller di method `dispose()` milik State.

Setelah terhubung, kita bisa memakai `controller.text` (membaca atau mengubah isi), `controller.clear()` (mengosongkan), dan `controller.addListener(...)` (dipanggil setiap isi berubah).

Contoh kecil

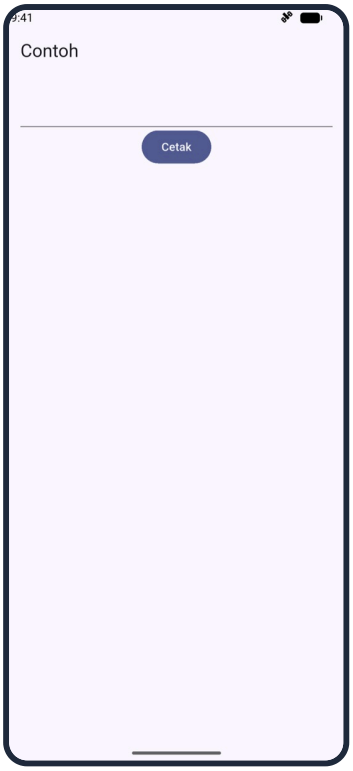
```
class ContohController extends StatefulWidget {
  const ContohController({super.key});

  @override
  State<ContohController> createState() => _ContohControllerState();
}

class _ContohControllerState extends State<ContohController> {
  // 1. Buat controller sebagai field di State
  final _namaController = TextEditingController();

  // 3. Wajib dibuang saat widget dihapus dari layar
  @override
  void dispose() {
    _namaController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        // 2. Hubungkan controller ke TextField
        TextField(controller: _namaController),
        FilledButton(
          // Baca isi TextField kapan saja lewat .text
          onPressed: () => debugPrint('Nama: ${_namaController.text}'),
          child: const Text('Cetak'),
        ),
      ],
    );
  }
}
```



TextField terhubung controller dan tombol Cetak

Contoh lengkap: membaca, mengubah, mengosongkan, dan mendengarkan

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ControllerDemoScreen(),
    );
  }
}

class ControllerDemoScreen extends StatefulWidget {
  const ControllerDemoScreen({super.key});

  @override
  State<ControllerDemoScreen> createState() => _ControllerDemoScreenState();
}

class _ControllerDemoScreenState extends State<ControllerDemoScreen> {
```

```dart
// Nilai awal bisa diisi lewat parameter text
final _judulController = TextEditingController(text: 'Rancang Bangun Aplikasi');
final List<String> _riwayat = [];
int _jumlahKata = 0;

@override
void initState() {
  super.initState();
  _jumlahKata = _hitung(_judulController.text);
  // Listener dipanggil SETIAP kali teks (atau posisi kursor) berubah
  _judulController.addListener(_hitungKata);
}

int _hitung(String teks) {
  final bersih = teks.trim();
  return bersih.isEmpty ? 0 : bersih.split(RegExp(r'\s+')).length;
}

void _hitungKata() {
  setState(() => _jumlahKata = _hitung(_judulController.text));
}

@override
void dispose() {
  _judulController.removeListener(_hitungKata);
  _judulController.dispose();
  super.dispose();
}

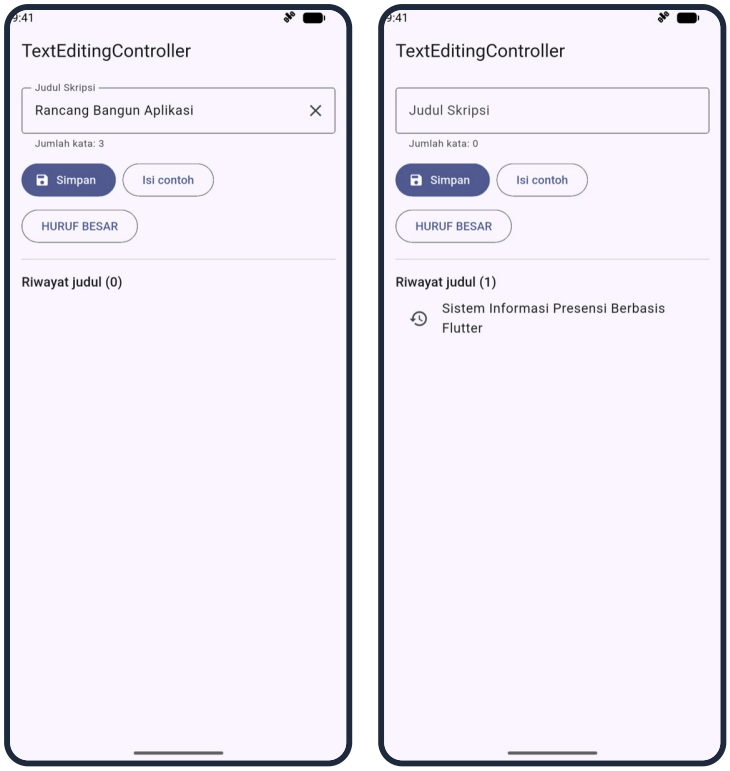
void _simpan() {
  final judul = _judulController.text.trim();
  if (judul.isEmpty) return;
  setState(() => _riwayat.insert(0, judul));
  _judulController.clear(); // mengosongkan TextField
}

void _isiContoh() {
  // Mengubah isi TextField dari kode
  _judulController.text = 'Sistem Informasi Presensi Berbasis Flutter';
}

void _hurufBesar() {
  _judulController.text = _judulController.text.toUpperCase();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('TextEditingController')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          TextField(
            controller: _judulController,
            decoration: InputDecoration(
              labelText: 'Judul Skripsi',
              border: const OutlineInputBorder(),
```

```
                helperText: 'Jumlah kata: $_jumlahKata',
                // Tombol hapus hanya muncul jika ada teks
                suffixIcon: _judulController.text.isEmpty
                    ? null
                    : IconButton(
                        icon: const Icon(Icons.clear),
                        onPressed: _judulController.clear,
                      ),
              ),
            ),
            const SizedBox(height: 12),
            Wrap(
              spacing: 8,
              runSpacing: 8,
              children: [
                FilledButton.icon(
                  onPressed: _simpan,
                  icon: const Icon(Icons.save),
                  label: const Text('Simpan'),
                ),
                OutlinedButton(onPressed: _isiContoh, child: const Text('Isi contoh')),
                OutlinedButton(onPressed: _hurufBesar, child: const Text('HURUF BESAR')),
              ],
            ),
            const Divider(height: 32),
            Text('Riwayat judul (${_riwayat.length})',
                style: Theme.of(context).textTheme.titleMedium),
            Expanded(
              child: ListView(
                children: [
                  for (final judul in _riwayat)
                    ListTile(leading: const Icon(Icons.history), title: Text(judul)),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```



Nilai awal dari TextEditingController(text: ...) dan jumlah kata

Setelah Isi contoh, Simpan, lalu field kosong kembali

Penjelasan kode:

- `TextEditingController(text: '...')` memberi isi awal pada field.
- `addListener(_hitungKata)` di `initState` membuat `_hitungKata` dipanggil setiap teks berubah, baik karena diketik maupun diubah dari kode. Inilah bedanya dengan `onChanged`, yang hanya terpanggil saat pengguna mengetik.
- `_judulController.text = '...'` mengganti isi field dari kode (tombol "Isi contoh" dan "HURUF BESAR").
- `_judulController.clear()` mengosongkan field setelah disimpan. Tombol silang di `suffixIcon` juga memanggil `clear`.
- Di `dispose()` kita melepas listener lalu membuang controller. Urutannya: bersihkan milik kita dulu, `super.dispose()` paling akhir.

	`onChanged`	`TextEditingController`
Membaca teks	Ya, lewat parameter callback	Ya, kapan saja lewat `.text`
Mengubah / mengosongkan dari kode	Tidak bisa	Bisa
Perlu `dispose`	Tidak	Ya, wajib
Cocok untuk	Pencarian sederhana, preview langsung	Form yang dikirim, nilai awal, tombol hapus

⚠️ **Kesalahan umum**

- **Membuat controller di dalam `build()`.** Setiap kali `setState` dipanggil, controller baru dibuat sehingga teks yang diketik hilang. Selalu jadikan field di State.
- **Lupa `dispose()`.** Controller tetap tersimpan di memori setelah layar ditutup (*memory leak*).
- **Memakai controller di StatelessWidget.** StatelessWidget tidak punya `dispose`, jadi controller harus berada di StatefulWidget.
- **Memanggil `setState` di `initState` sebelum layar tampil.** Cukup isi variabel langsung, seperti `_jumlahKata = _hitung(...)` pada contoh.

5. Jenis Keyboard, Password & FocusNode

Form yang nyaman dipakai di HP bukan hanya soal tampilan. Jenis keyboard yang tepat, password yang bisa diintip, dan perpindahan fokus otomatis membuat pengguna bisa mengisi form lebih cepat.

1. keyboardType: memilih jenis keyboard

Parameter `keyboardType` menentukan keyboard yang muncul. Ini tidak membatasi isi (pengguna tetap bisa menempelkan huruf), tapi sangat memudahkan pengisian.

```
TextField(
  keyboardType: TextInputType.emailAddress, // keyboard dengan tombol @
  decoration: InputDecoration(labelText: 'Email'),
),
TextField(
  keyboardType: TextInputType.number, // keyboard angka
  decoration: InputDecoration(labelText: 'IPK Semester Lalu'),
),
TextField(
  keyboardType: TextInputType.phone, // keyboard telepon
  decoration: InputDecoration(labelText: 'Nomor WhatsApp'),
),
TextField(
  keyboardType: TextInputType.multiline, // Enter = baris baru
  maxLines: null,
  decoration: InputDecoration(labelText: 'Catatan'),
),
```



Field IPK sedang fokus; field ini memakai TextInputType.number sehingga HP menampilkan keyboard angka

Nilai	Keyboard	Untuk field
`TextInputType.text`	Huruf (bawaan)	Nama, alamat
`TextInputType.emailAddress`	Huruf + tombol @ dan .	Email
`TextInputType.number`	Angka	NIM, umur, nilai
`TextInputType.phone`	Papan telepon	Nomor HP
`TextInputType.url`	Huruf + tombol / dan .com	Alamat website
`TextInputType.multiline`	Enter membuat baris baru	Catatan, deskripsi

2. obscureText: password dengan tombol tampilkan/sembunyikan

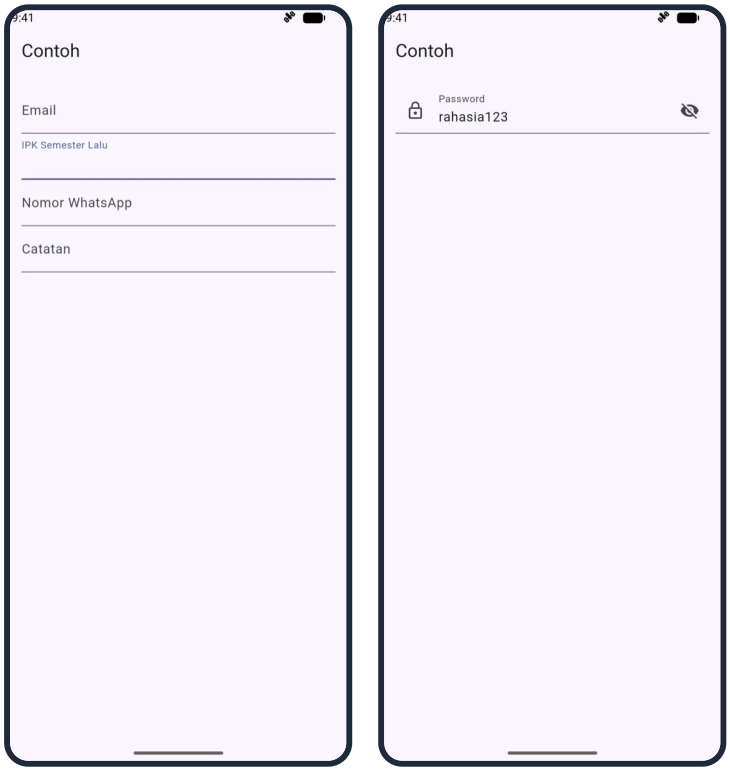
`obscureText: true` mengganti setiap huruf dengan titik. Agar pengguna bisa memeriksa ketikannya, kita simpan nilai `obscureText` di variabel State lalu membaliknya lewat `IconButton` di `suffixIcon`.

```
class PasswordField extends StatefulWidget {
  const PasswordField({super.key});
```

```dart
  @override
  State<PasswordField> createState() => _PasswordFieldState();
}

class _PasswordFieldState extends State<PasswordField> {
  bool _sembunyikan = true; // awalnya password disembunyikan

  @override
  Widget build(BuildContext context) {
    return TextField(
      obscureText: _sembunyikan, // true -> teks tampil sebagai titik
      enableSuggestions: false,
      autocorrect: false,
      decoration: InputDecoration(
        labelText: 'Password',
        prefixIcon: const Icon(Icons.lock_outline),
        suffixIcon: IconButton(
          tooltip: _sembunyikan ? 'Tampilkan password' : 'Sembunyikan password',
          icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
          onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
        ),
      ),
    );
  }
}
```



Password tersembunyi (titik-titik)

Setelah ikon mata ditekan, password terlihat

`enableSuggestions: false` dan `autocorrect: false` mencegah keyboard menyimpan atau mengoreksi password.

3. textInputAction dan FocusNode: berpindah antar field

- `textInputAction` mengganti tombol Enter di keyboard menjadi **Next** (`TextInputAction.next`), **Done** (`.done`), **Search** (`.search`), dan lainnya.
- `onSubmitted` dipanggil saat tombol tersebut ditekan.
- `FocusNode` adalah "remote" untuk sebuah field. `focusNode.requestFocus()` memindahkan kursor ke field itu, sedangkan `FocusScope.of(context).unfocus()` melepas fokus dan menutup keyboard. Sama seperti controller, FocusNode wajib di-`dispose`.

Contoh lengkap: login dengan perpindahan fokus

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const LoginSederhanaScreen(),
    );
  }
}

class LoginSederhanaScreen extends StatefulWidget {
  const LoginSederhanaScreen({super.key});

  @override
  State<LoginSederhanaScreen> createState() => _LoginSederhanaScreenState();
}

class _LoginSederhanaScreenState extends State<LoginSederhanaScreen> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  // FocusNode = "remote" untuk mengatur fokus sebuah TextField
  final _emailFocus = FocusNode();
  final _passwordFocus = FocusNode();

  bool _sembunyikan = true;
  String _status = 'Silakan login';

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    _emailFocus.dispose();
```

```
    _passwordFocus.dispose();
    super.dispose();
  }

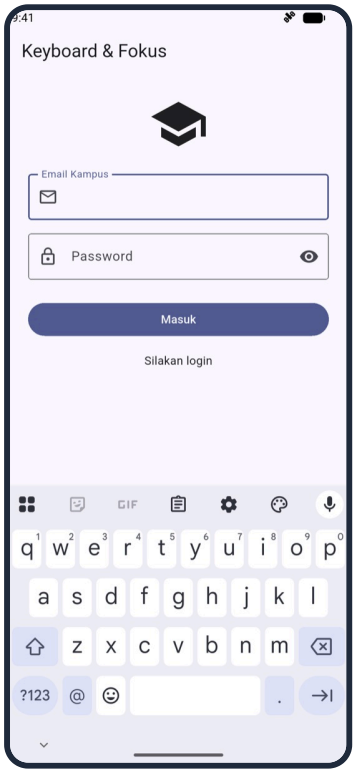
  void _login() {
    // Tutup keyboard dengan melepas fokus
    FocusScope.of(context).unfocus();

    final email = _emailController.text.trim();
    final password = _passwordController.text;

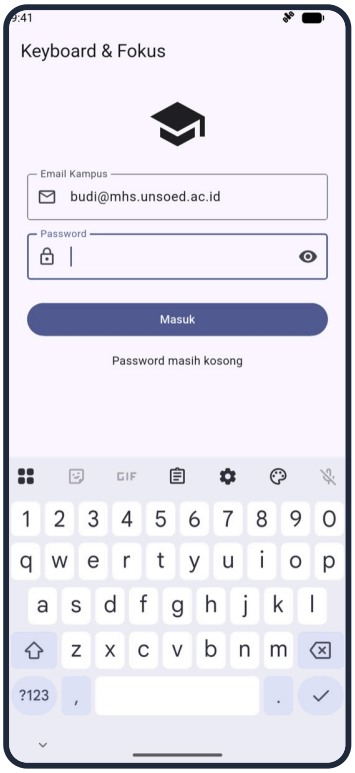
    if (email.isEmpty) {
      setState(() => _status = 'Email masih kosong');
      _emailFocus.requestFocus(); // arahkan kursor ke field yang bermasalah
      return;
    }
    if (password.isEmpty) {
      setState(() => _status = 'Password masih kosong');
      _passwordFocus.requestFocus();
      return;
    }
    setState(() => _status = 'Login sebagai $email');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keyboard & Fokus')),
      // Ketuk area kosong untuk menutup keyboard
      body: GestureDetector(
        onTap: () => FocusScope.of(context).unfocus(),
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            const Icon(Icons.school, size: 72),
            const SizedBox(height: 24),
            TextField(
              controller: _emailController,
              focusNode: _emailFocus,
              autofocus: true, // langsung fokus saat layar dibuka
              keyboardType: TextInputType.emailAddress,
              textInputAction: TextInputAction.next, // tombol "Next" di keyboard
              // Saat "Next" ditekan, pindah ke field password
              onSubmitted: (_) => _passwordFocus.requestFocus(),
              decoration: const InputDecoration(
                labelText: 'Email Kampus',
                prefixIcon: Icon(Icons.email_outlined),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            TextField(
              controller: _passwordController,
              focusNode: _passwordFocus,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.done, // tombol "Done" di keyboard
              onSubmitted: (_) => _login(), // tekan Done = login
              decoration: InputDecoration(
                labelText: 'Password',
```

```
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
                  onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
                ),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _login, child: const Text('Masuk')),
            const SizedBox(height: 16),
            Text(_status, textAlign: TextAlign.center),
          ],
        ),
      ),
    );
  }
}
```



Field email langsung fokus (autofocus) dan keyboard email terbuka



Email diisi, Masuk ditekan: kursor dipindah ke Password

Penjelasan kode:

- `autofocus: true` pada email membuat keyboard langsung muncul saat layar dibuka.
- Pada email, `onSubmitted: (_) => _passwordFocus.requestFocus()` memindahkan kursor ke password ketika tombol Next ditekan.
- Pada password, `TextInputAction.done` + `onSubmitted: (_) => _login()` membuat pengguna bisa login langsung dari keyboard.
- Di `_login`, jika ada field kosong, kursor diarahkan ke field tersebut dengan `requestFocus()`.

- `GestureDetector` di luar `ListView` menutup keyboard saat area kosong diketuk.

> ⚠️ **Kesalahan umum**
>
> - **Membuat `FocusNode()` di dalam `build`.** Sama seperti controller: buat di State dan dispose.
> - **Memberi `obscureText: true` pada field multi-baris.** Flutter menolak kombinasi ini (`maxLines` harus 1).
> - **Mengandalkan `keyboardType.number` sebagai validasi.** Pengguna tetap bisa menempelkan huruf. Validasi tetap dilakukan di `validator` (section berikutnya).

6. Form, TextFormField & Validasi

Memeriksa setiap field satu per satu dengan `if` cepat menjadi berantakan. Flutter menyediakan `Form`, `TextFormField`, dan `GlobalKey<FormState>` untuk memvalidasi semua field sekaligus. Pola di section ini akan kita pakai lagi untuk layar login dan register pada Modul 13-14.

Konsep: empat bagian

1. `GlobalKey<FormState>`: "kunci" untuk mengakses form dari luar, misalnya dari tombol Kirim.
2. `Form`: pembungkus semua field.
3. `TextFormField`: seperti TextField tetapi punya parameter `validator`.
4. `_formKey.currentState!.validate()`: menjalankan semua validator. Hasilnya `true` jika semua valid.

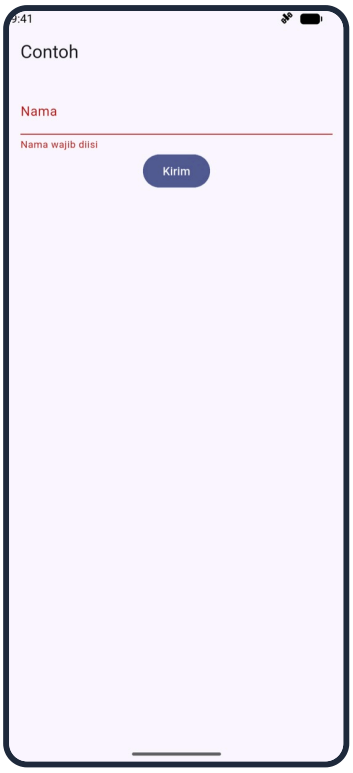
Aturan validator: **kembalikan `null` jika valid**, atau kembalikan `String` berisi pesan error jika tidak valid. Pesan itu otomatis tampil merah di bawah field.

Contoh kecil

```dart
// 1. Kunci untuk mengakses FormState
final _formKey = GlobalKey<FormState>();

@override
Widget build(BuildContext context) {
  // 2. Bungkus semua field dengan Form
  return Form(
    key: _formKey,
    child: Column(
      children: [
        // 3. Pakai TextFormField (bukan TextField) agar punya validator
        TextFormField(
          decoration: const InputDecoration(labelText: 'Nama'),
          validator: (value) {
            if (value == null || value.trim().isEmpty) {
              return 'Nama wajib diisi'; // String = pesan error
            }
            return null; // null = valid
          },
        ),
        FilledButton(
          onPressed: () {
            // 4. Jalankan SEMUA validator sekaligus
            if (_formKey.currentState!.validate()) {
              debugPrint('Form valid, data boleh diproses');
            }
          },
          child: const Text('Kirim'),
        ),
      ],
    ),
```

```
    );
  }
```



Setelah Kirim ditekan dengan field kosong

Mengumpulkan aturan dalam kelas Validators

Aturan yang sama (email, password) akan dipakai di banyak layar. Lebih rapi jika dikumpulkan dalam satu kelas berisi fungsi `static`. Pada proyek nyata, kelas ini bisa disimpan di `lib/utils/validators.dart`.

```dart
class Validators {
  // Pola email sederhana: teks@teks.domain
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? value, String namaField) {
    if (value == null || value.trim().isEmpty) {
      return '$namaField wajib diisi';
    }
    return null;
  }

  static String? email(String? value) {
    if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(value.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value, {int minPanjang = 8}) {
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < minPanjang) return 'Password minimal $minPanjang karakter';
    if (!RegExp(r'[0-9]').hasMatch(value)) return 'Password harus mengandung angka';
    return null;
```

```
  }

  static String? konfirmasi(String? value, String passwordAsli) {
    if (value == null || value.isEmpty) return 'Ulangi password';
    if (value != passwordAsli) return 'Konfirmasi password tidak sama';
    return null;
  }
}
```

- `RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$')` berarti: satu atau lebih huruf/angka/titik/plus/strip, lalu `@`, lalu nama domain, lalu minimal satu bagian `.xxx`. Awalan `r` membuat string "mentah" sehingga `\` tidak perlu ditulis dua kali.
- `konfirmasi` menerima dua nilai: isian konfirmasi dan password asli. Keduanya dibandingkan dengan `!=`.

Contoh lengkap: form registrasi akun

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const RegisterScreen(),
    );
  }
}

// ===== Kumpulan validator (nanti bisa dipindah ke lib/utils/validators.dart) =====
class Validators {
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? value, String namaField) {
    if (value == null || value.trim().isEmpty) return '$namaField wajib diisi';
    return null;
  }

  static String? email(String? value) {
    if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(value.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value, {int minPanjang = 8}) {
```

```
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < minPanjang) return 'Password minimal $minPanjang karakter';
    if (!RegExp(r'[0-9]').hasMatch(value)) return 'Password harus mengandung angka';
    return null;
  }

  static String? konfirmasi(String? value, String passwordAsli) {
    if (value == null || value.isEmpty) return 'Ulangi password';
    if (value != passwordAsli) return 'Konfirmasi password tidak sama';
    return null;
  }
}

// ===== Layar registrasi =====
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _namaController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _konfirmasiController = TextEditingController();

  bool _sembunyikanPassword = true;
  bool _sedangMengirim = false;

  @override
  void dispose() {
    _namaController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _konfirmasiController.dispose();
    super.dispose();
  }

  Future<void> _daftar() async {
    // Tutup keyboard lalu validasi semua field
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

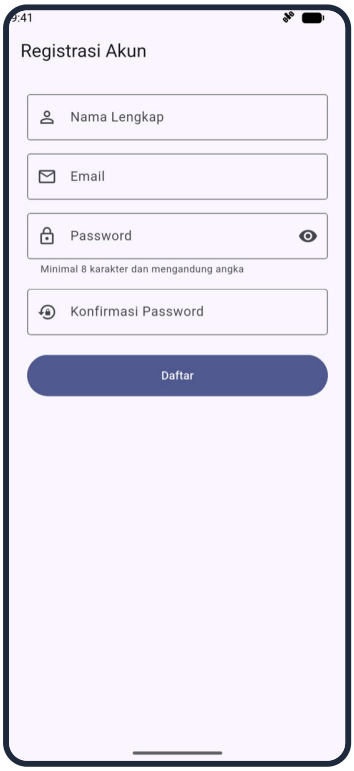
    setState(() => _sedangMengirim = true);
    // Simulasi mengirim data ke server selama 2 detik
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return; // widget bisa saja sudah ditutup selama menunggu

    setState(() => _sedangMengirim = false);
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Akun ${_emailController.text.trim()} berhasil dibuat')),
    );
    _formKey.currentState!.reset(); // kosongkan error
    _namaController.clear();
    _emailController.clear();
    _passwordController.clear();
    _konfirmasiController.clear();
```

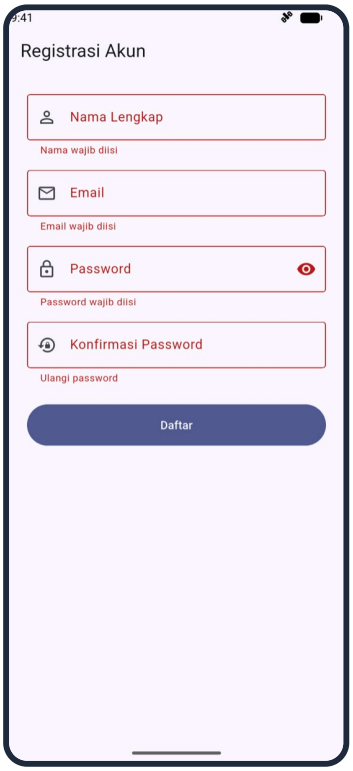
```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Registrasi Akun')),
      body: Form(
        key: _formKey,
        // Validasi ulang otomatis setelah pengguna mulai berinteraksi
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _namaController,
              textInputAction: TextInputAction.next,
              textCapitalization: TextCapitalization.words,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person_outline),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.wajib(v, 'Nama'),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _emailController,
              keyboardType: TextInputType.emailAddress,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Email',
                hintText: 'nama@mhs.unsoed.ac.id',
                prefixIcon: Icon(Icons.email_outlined),
                border: OutlineInputBorder(),
              ),
              validator: Validators.email,
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _passwordController,
              obscureText: _sembunyikanPassword,
              textInputAction: TextInputAction.next,
              decoration: InputDecoration(
                labelText: 'Password',
                helperText: 'Minimal 8 karakter dan mengandung angka',
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikanPassword ? Icons.visibility : Icons.visibility_off),
                  onPressed: () =>
                      setState(() => _sembunyikanPassword = !_sembunyikanPassword),
                ),
              ),
              validator: (v) => Validators.password(v),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _konfirmasiController,
              obscureText: _sembunyikanPassword,
```

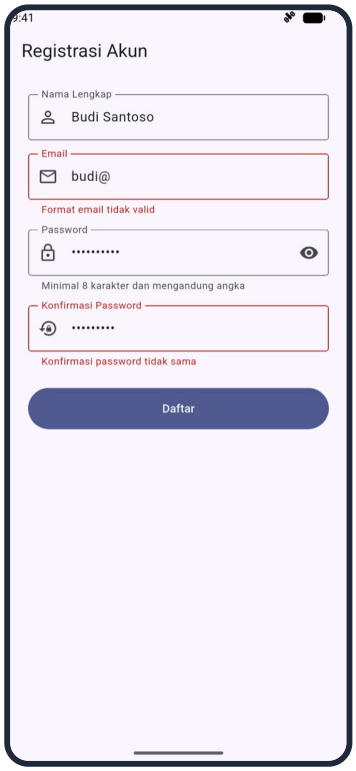
```
              textInputAction: TextInputAction.done,
              onFieldSubmitted: (_) => _daftar(),
              decoration: const InputDecoration(
                labelText: 'Konfirmasi Password',
                prefixIcon: Icon(Icons.lock_reset),
                border: OutlineInputBorder(),
              ),
              // Bandingkan dengan isi controller password
              validator: (v) => Validators.konfirmasi(v, _passwordController.text),
            ),
            const SizedBox(height: 24),
            FilledButton(
              // null saat mengirim -> tombol disabled, cegah klik ganda
              onPressed: _sedangMengirim ? null : _daftar,
              style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(50)),
              child: _sedangMengirim
                  ? const SizedBox(
                      width: 22,
                      height: 22,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Daftar'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Form registrasi sebelum diisi



Daftar ditekan saat form kosong: semua pesan error tampil



Email salah format dan konfirmasi password tidak sama

Penjelasan kode:

- `validator: Validators.email` bisa ditulis tanpa lambda karena tipe fungsinya sama persis dengan yang diminta (`String? Function(String?)`). Untuk fungsi dengan parameter tambahan, pakai lambda: `(v) => Validators.wajib(v, 'Nama')`.
- Validator konfirmasi membaca `_passwordController.text`, inilah alasan password perlu controller.
- `autovalidateMode: AutovalidateMode.onUserInteraction` membuat pesan error diperbarui sambil mengetik, tetapi hanya setelah field pernah disentuh. Pengguna tidak langsung "dimarahi" saat layar baru dibuka.
- Selama `_sedangMengirim` bernilai `true`, `onPressed` menjadi `null` (tombol nonaktif) dan isi tombol diganti `CircularProgressIndicator`. Ini mencegah data terkirim dua kali.
- `if (!mounted) return;` setelah `await` memastikan layar masih ada sebelum memakai `context` atau `setState`.
- `_formKey.currentState!.reset()` menghapus pesan error; controller dikosongkan dengan `clear()`.

Contoh lengkap kedua: form KRS dengan Dropdown dan Checkbox

Contoh ini memperlihatkan bahwa `DropdownButtonFormField` juga punya `validator` dan ikut diperiksa oleh `validate()`. Checkbox tidak punya validator, sehingga diperiksa manual dengan `if`.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text("Formulir KRS Mahasiswa")),
        // PENTING: TextFormField, Checkbox & SnackBar butuh Scaffold/Material sebagai induk.
        // Jika FormKrsMahasiswa dipasang langsung sebagai home tanpa Scaffold, akan muncul
        // error "No Material widget found" (layar merah / kosong).
        body: const FormKrsMahasiswa(),
      ),
    );
  }
}


class FormKrsMahasiswa extends StatefulWidget {
```

```
  const FormKrsMahasiswa({super.key});

  @override
  State<FormKrsMahasiswa> createState() => _FormKrsMahasiswaState();
}

class _FormKrsMahasiswaState extends State<FormKrsMahasiswa> {
  // 1. Kunci Global untuk mengontrol dan memvalidasi FormState
  final _formKey = GlobalKey<FormState>();

  // 2. Controller untuk membaca teks dari input field
  final TextEditingController _nimController = TextEditingController();
  final TextEditingController _namaController = TextEditingController();

  String? _prodiTerpilih;
  bool _setujuSyarat = false;

  final List<String> _daftarProdi = [
    'Teknik Informatika',
    'Sistem Informasi',
    'Teknologi Rekayasa Perangkat Lunak',
    'Sains Data',
  ];

  // 3. WAJIB dispose controller agar tidak terjadi memory leak!
  @override
  void dispose() {
    _nimController.dispose();
    _namaController.dispose();
    super.dispose();
  }

  void _submitForm() {
    // Mengeksekusi seluruh fungsi validator pada setiap FormField di dalam Form
    if (_formKey.currentState!.validate()) {
      if (!_setujuSyarat) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text("Anda harus menyetujui syarat & ketentuan KRS!"),
            backgroundColor: Colors.red,
          ),
        );
        return;
      }

      // Jika lolos validasi, proses data
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text("Data Mahasiswa ${_namaController.text} (${_nimController.text}) Berhasil
Diverifikasi!"),
          backgroundColor: Colors.green,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
```

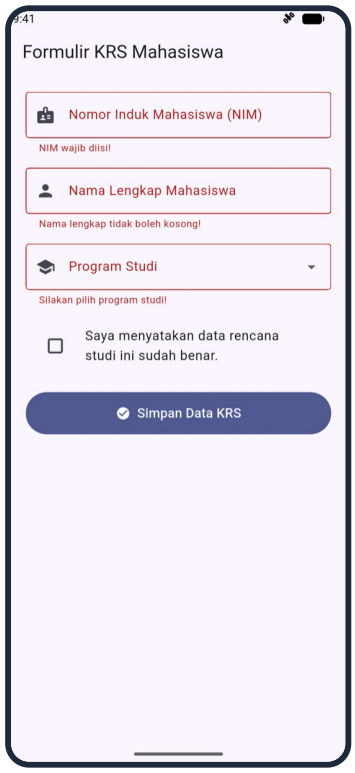
```
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          // Input NIM dengan Validasi Regex
          TextFormField(
            controller: _nimController,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
              labelText: "Nomor Induk Mahasiswa (NIM)",
              hintText: "Contoh: 21102001",
              prefixIcon: Icon(Icons.badge),
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return "NIM wajib diisi!";
              }
              if (!RegExp(r'^[0-9]{8,}$').hasMatch(value.trim())) {
                return "NIM harus berupa angka minimal 8 digit!";
              }
              return null; // Nilai null berarti VALID
            },
          ),
          const SizedBox(height: 16),

          // Input Nama Lengkap
          TextFormField(
            controller: _namaController,
            decoration: const InputDecoration(
              labelText: "Nama Lengkap Mahasiswa",
              prefixIcon: Icon(Icons.person),
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return "Nama lengkap tidak boleh kosong!";
              }
              return null;
            },
          ),
          const SizedBox(height: 16),

          // Input Dropdown Pilihan Program Studi
          DropdownButtonFormField<String>(
            initialValue: _prodiTerpilih, // Flutter < 3.35: gunakan `value:`
            isExpanded: true, // Wajib agar nama prodi yang panjang tidak overflow di layar HP kecil
            decoration: const InputDecoration(
              labelText: "Program Studi",
              prefixIcon: Icon(Icons.school),
              border: OutlineInputBorder(),
            ),
            items: _daftarProdi.map((prodi) {
              return DropdownMenuItem(value: prodi, child: Text(prodi));
            }).toList(),
            onChanged: (val) => setState(() => _prodiTerpilih = val),
            validator: (val) => val == null ? "Silakan pilih program studi!" : null,
          ),
          const SizedBox(height: 16),
```

```
          // Checkbox Persetujuan
          CheckboxListTile(
            title: const Text("Saya menyatakan data rencana studi ini sudah benar."),
            value: _setujuSyarat,
            onChanged: (val) => setState(() => _setujuSyarat = val ?? false),
            controlAffinity: ListTileControlAffinity.leading,
          ),
          const SizedBox(height: 24),

          // Tombol Submit
          FilledButton.icon(
            onPressed: _submitForm,
            icon: const Icon(Icons.check_circle),
            label: const Text("Simpan Data KRS", style: TextStyle(fontSize: 16)),
            style: FilledButton.styleFrom(padding: const EdgeInsets.symmetric(vertical: 14)),
          ),
        ],
      ),
    );
  }
}
```



Form KRS setelah tombol Simpan ditekan tanpa isian

ℹ️ onSaved dan save()

Selain `validator`, TextFormField punya `onSaved` yang dipanggil oleh `_formKey.currentState!.save()`. Ini alternatif dari controller: setiap field menyimpan nilainya ke variabel saat `save()` dipanggil. Pada modul ini kita memakai controller karena lebih mudah dibaca dan bisa dipakai untuk mengosongkan field.

⚠️ Kesalahan umum

- **Validator mengembalikan `""` (string kosong) untuk input valid.** String kosong tetap dianggap error. Yang benar `return null;`.
- **Memakai `TextField` di dalam Form.** TextField tidak punya validator. Gunakan `TextFormField`.
- **Membuat `GlobalKey` di dalam `build`.** Key baru di setiap build membuat form kehilangan state. Jadikan field di State.
- **Lupa `trim()`.** Isian berupa spasi saja akan lolos pemeriksaan `isEmpty`.
- **Error "No Material widget found".** Form harus berada di dalam `Scaffold`.

7. Checkbox, Switch, Radio & Slider

Tidak semua input berupa ketikan. Untuk pilihan ya/tidak, satu dari beberapa opsi, atau nilai dalam rentang, Flutter menyediakan Checkbox, Switch, Radio, dan Slider. Semuanya bekerja dengan pola yang sama: **nilai disimpan di State, ditampilkan lewat `value`, dan diubah lewat `onChanged` + `setState`.**

Widget	Tipe nilai	Kapan dipakai
`Checkbox` / `CheckboxListTile`	`bool`	Persetujuan, boleh memilih banyak opsi
`Switch` / `SwitchListTile`	`bool`	Pengaturan on/off yang langsung berlaku
`RadioGroup` + `RadioListTile`	Satu nilai (sering `enum`)	Pilih tepat satu dari beberapa opsi
`Slider` / `RangeSlider`	`double` / `RangeValues`	Nilai atau rentang angka

1. Checkbox dan Switch

Versi `...ListTile` sudah termasuk judul, dan seluruh barisnya bisa diketuk. Hampir selalu lebih nyaman dipakai daripada versi polos.

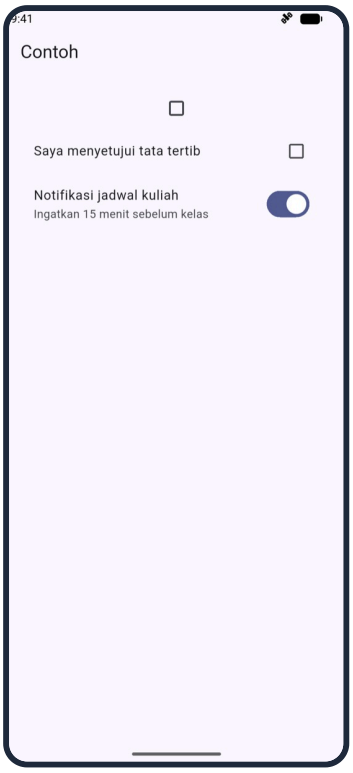
```
bool _setuju = false;
bool _notifikasi = true;

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      // Checkbox polos: hanya kotaknya saja
      Checkbox(
        value: _setuju,
        onChanged: (baru) => setState(() => _setuju = baru ?? false),
      ),

      // CheckboxListTile: kotak + judul, seluruh baris bisa diketuk
      CheckboxListTile(
        title: const Text('Saya menyetujui tata tertib'),
        value: _setuju,
        onChanged: (baru) => setState(() => _setuju = baru ?? false),
      ),

      // SwitchListTile: saklar on/off + judul
      SwitchListTile(
        title: const Text('Notifikasi jadwal kuliah'),
        subtitle: const Text('Ingatkan 15 menit sebelum kelas'),
```

```
        value: _notifikasi,
        onChanged: (baru) => setState(() => _notifikasi = baru),
      ),
    ],
  );
}
```



Checkbox polos, CheckboxListTile, dan SwitchListTile

Parameter `onChanged` Checkbox menerima `bool?` (bisa null untuk mode tiga keadaan), sehingga kita menulis `baru ?? false`. Switch menerima `bool` biasa.

2. Radio dengan RadioGroup

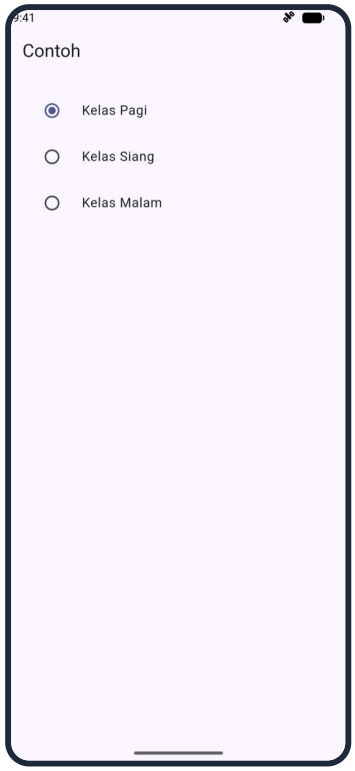
Sejak Flutter 3.35, nilai terpilih **tidak lagi** ditulis di setiap Radio (`groupValue` dan `onChanged` di Radio sudah *deprecated*). Sekarang semua radio dibungkus satu `RadioGroup` yang memegang `groupValue` dan `onChanged`, dan setiap radio cukup diberi `value`.

```
enum Kelas { pagi, siang, malam }

class _ContohState extends State<Contoh> {
  Kelas? _kelas = Kelas.pagi;

  @override
  Widget build(BuildContext context) {
    // RadioGroup menyimpan nilai terpilih untuk SEMUA radio di dalamnya
    return RadioGroup<Kelas>(
      groupValue: _kelas,
      onChanged: (baru) => setState(() => _kelas = baru),
      child: const Column(
        children: [
          // Tiap radio cukup diberi value-nya masing-masing
```

```
          RadioListTile<Kelas>(value: Kelas.pagi, title: Text('Kelas Pagi')),
          RadioListTile<Kelas>(value: Kelas.siang, title: Text('Kelas Siang')),
          RadioListTile<Kelas>(value: Kelas.malam, title: Text('Kelas Malam')),
        ],
      ),
    );
  }
}
```



RadioGroup dengan tiga pilihan kelas

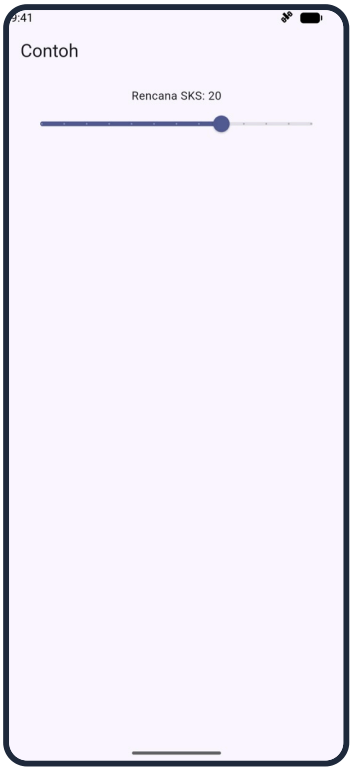
> ℹ️ **Kenapa memakai enum?**
>
> Dengan `enum Kelas { pagi, siang, malam }`, salah ketik seperti `'Pagi'` vs `'pagi'` tidak mungkin terjadi, karena compiler langsung memberi tahu jika nilai tidak dikenal. Nama enum bisa ditampilkan dengan `kelas.name`.

3. Slider

```
double _sks = 20;

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text('Rencana SKS: ${_sks.round()}'),
      Slider(
        value: _sks,
```

```
        min: 12,
        max: 24,
        divisions: 12, // 12 langkah -> nilainya bulat 12, 13, ..., 24
        label: '${_sks.round()} SKS', // gelembung saat digeser
        onChanged: (baru) => setState(() => _sks = baru),
      ),
    ],
  );
}
```



Slider rencana SKS

Tanpa `divisions`, nilai Slider berupa pecahan (misalnya 17.3826). Dengan `divisions`, nilai melompat per langkah, dan `label` menampilkan gelembung nilai saat digeser.

Contoh lengkap: halaman preferensi belajar

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const PreferensiScreen(),
    );
  }
}

enum ModeBelajar { luring, daring, hybrid }

class PreferensiScreen extends StatefulWidget {
  const PreferensiScreen({super.key});

  @override
  State<PreferensiScreen> createState() => _PreferensiScreenState();
}

class _PreferensiScreenState extends State<PreferensiScreen> {
  // Checkbox: boleh memilih lebih dari satu
  final Map<String, bool> _minat = {
    'Mobile Development': true,
    'Web Development': false,
    'Data Science': false,
    'Jaringan Komputer': false,
  };

  // Switch: on/off
  bool _notifikasi = true;
  bool _modeGelap = false;

  // Radio: hanya satu pilihan
  ModeBelajar _mode = ModeBelajar.luring;

  // Slider: nilai dalam rentang
  double _targetIpk = 3.5;
  RangeValues _jamBelajar = const RangeValues(19, 22);

  String get _ringkasan {
    final minatDipilih =
        _minat.entries.where((e) => e.value).map((e) => e.key).join(', ');
    return 'Minat: ${minatDipilih.isEmpty ? '-' : minatDipilih}\n'
        'Notifikasi: ${_notifikasi ? 'aktif' : 'mati'}, '
        'Mode gelap: ${_modeGelap ? 'ya' : 'tidak'}\n'
        'Mode belajar: ${_mode.name}\n'
        'Target IPK: ${_targetIpk.toStringAsFixed(2)}\n'
        'Jam belajar: ${_jamBelajar.start.round()}.00 - ${_jamBelajar.end.round()}.00';
  }

  @override
  Widget build(BuildContext context) {
    final judul = Theme.of(context).textTheme.titleMedium;

    return Scaffold(
      appBar: AppBar(title: const Text('Preferensi Belajar')),
      body: ListView(
        padding: const EdgeInsets.symmetric(vertical: 8),
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Bidang minat (boleh lebih dari satu)', style: judul),
          ),
```

```
          for (final nama in _minat.keys)
            CheckboxListTile(
              title: Text(nama),
              value: _minat[nama],
              onChanged: (baru) => setState(() => _minat[nama] = baru ?? false),
            ),
          const Divider(),

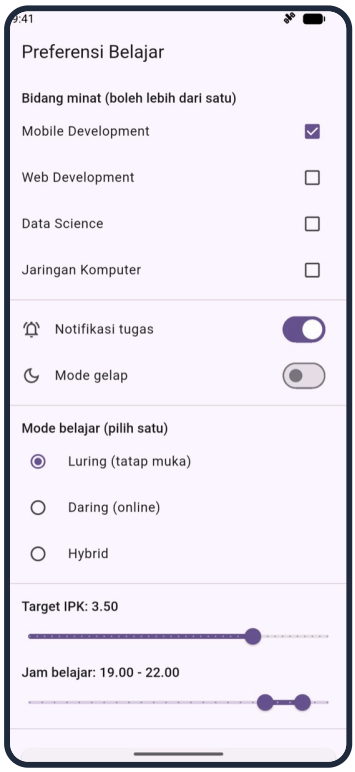
          SwitchListTile(
            secondary: const Icon(Icons.notifications_active_outlined),
            title: const Text('Notifikasi tugas'),
            value: _notifikasi,
            onChanged: (baru) => setState(() => _notifikasi = baru),
          ),
          SwitchListTile(
            secondary: const Icon(Icons.dark_mode_outlined),
            title: const Text('Mode gelap'),
            value: _modeGelap,
            onChanged: (baru) => setState(() => _modeGelap = baru),
          ),
          const Divider(),

          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Mode belajar (pilih satu)', style: judul),
          ),
          RadioGroup<ModeBelajar>(
            groupValue: _mode,
            onChanged: (baru) {
              if (baru != null) setState(() => _mode = baru);
            },
            child: const Column(
              children: [
                RadioListTile(value: ModeBelajar.luring, title: Text('Luring (tatap muka)')),
                RadioListTile(value: ModeBelajar.daring, title: Text('Daring (online)')),
                RadioListTile(value: ModeBelajar.hybrid, title: Text('Hybrid')),
              ],
            ),
          ),
          const Divider(),

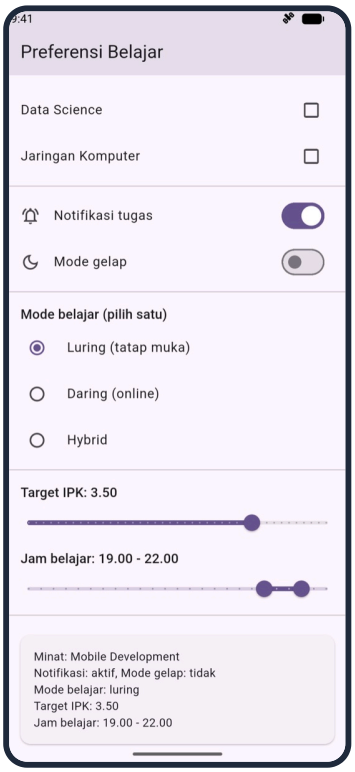
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Target IPK: ${_targetIpk.toStringAsFixed(2)}', style: judul),
          ),
          Slider(
            value: _targetIpk,
            min: 2.0,
            max: 4.0,
            divisions: 40, // langkah 0.05
            label: _targetIpk.toStringAsFixed(2),
            onChanged: (baru) => setState(() => _targetIpk = baru),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text(
              'Jam belajar: ${_jamBelajar.start.round()}.00 - ${_jamBelajar.end.round()}.00',
              style: judul,
            ),
```

```
          ),
          // RangeSlider: memilih rentang (dua titik)
          RangeSlider(
            values: _jamBelajar,
            min: 0,
            max: 24,
            divisions: 24,
            labels: RangeLabels(
              '${_jamBelajar.start.round()}.00',
              '${_jamBelajar.end.round()}.00',
            ),
            onChanged: (baru) => setState(() => _jamBelajar = baru),
          ),
          const Divider(),

          Card(
            margin: const EdgeInsets.all(16),
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Text(_ringkasan),
            ),
          ),
        ],
      ),
    );
  }
}
```



Bagian atas: daftar Checkbox dan Switch



Setelah digulir: Radio, Slider, RangeSlider, dan ringkasan

Penjelasan kode:

- `Map<String, bool> _minat` menyimpan status setiap checkbox. `for (final nama in _minat.keys)` di dalam `children` membuat satu CheckboxListTile per minat.
- `RadioGroup<ModeBelajar>` membungkus `Column` berisi tiga `RadioListTile`. Karena RadioListTile di dalamnya tidak berubah, Column bisa `const`.
- `divisions: 40` pada rentang 2.0-4.0 menghasilkan langkah 0.05.
- `RangeSlider` memakai `RangeValues(start, end)` untuk dua titik sekaligus.
- Getter `_ringkasan` menyusun semua nilai menjadi teks. Karena dihitung ulang setiap build, ringkasan selalu sesuai pilihan terbaru.

> ⚠️ **Kesalahan umum**
>
> - **Lupa `setState` di `onChanged`.** Nilai variabel berubah tetapi tampilan tidak; checkbox terlihat "macet".
> - **Menulis `Radio(groupValue: ..., onChanged: ...)`.** Cara lama ini menghasilkan peringatan deprecated. Gunakan `RadioGroup`.
> - **Nilai awal Slider di luar `min`-`max`.** Menyebabkan error assertion. Pastikan `min <= value <= max`.
> - **Menyimpan nilai di variabel lokal dalam `build`.** Nilai akan kembali ke awal di setiap build. Simpan sebagai field State.

8. Dropdown, DatePicker & TimePicker

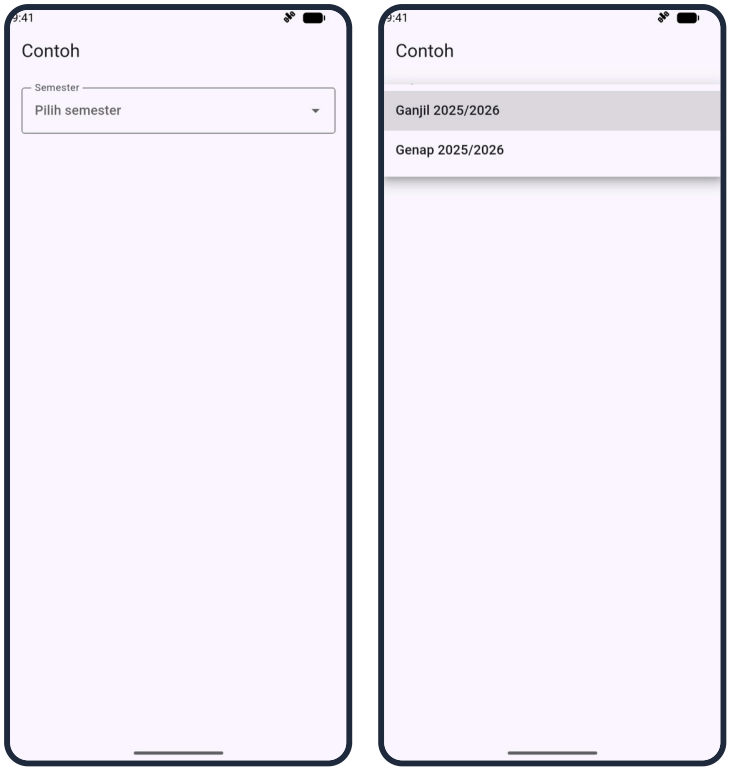
Untuk pilihan yang banyak (program studi, dosen) dan untuk tanggal/jam, pengguna sebaiknya memilih, bukan mengetik. Bagian ini membahas `DropdownButtonFormField`, `showDatePicker`, dan `showTimePicker`.

1. DropdownButtonFormField

Versi `...FormField` dipilih karena bisa memakai `InputDecoration` dan `validator` seperti TextFormField.

```dart
final List<String> _daftarSemester = ['Ganjil 2025/2026', 'Genap 2025/2026'];
String? _semester; // null = belum memilih

@override
Widget build(BuildContext context) {
  return DropdownButtonFormField<String>(
    initialValue: _semester, // nilai awal (Flutter < 3.35: pakai `value:`)
    hint: const Text('Pilih semester'),
    decoration: const InputDecoration(
      labelText: 'Semester',
      border: OutlineInputBorder(),
    ),
    items: _daftarSemester
        .map((s) => DropdownMenuItem(value: s, child: Text(s)))
        .toList(),
    onChanged: (baru) => setState(() => _semester = baru),
    validator: (v) => v == null ? 'Semester wajib dipilih' : null,
  );
}
```



Dropdown sebelum dipilih

Daftar pilihan setelah dropdown diketuk

- `items` adalah daftar `DropdownMenuItem`. Setiap item punya `value` (nilai yang disimpan) dan `child` (yang ditampilkan).
- `.map(...).toList()` mengubah `List<String>` menjadi `List<DropdownMenuItem>`.
- Nilai awal ditulis di `initialValue`. Pada Flutter sebelum 3.35 namanya `value`.

2. showDatePicker dan showTimePicker

Keduanya adalah fungsi `async` yang membuka dialog dan **mengembalikan nilai pilihan** setelah dialog ditutup, atau `null` jika pengguna menekan Batal.

```
DateTime? _tanggal;
TimeOfDay? _jam;

Future<void> _pilihTanggal() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggal ?? DateTime.now(),
    firstDate: DateTime(2020),
    lastDate: DateTime(2030),
    helpText: 'Pilih tanggal seminar',
  );
  // hasil == null jika pengguna menekan Batal
  if (hasil != null) setState(() => _tanggal = hasil);
}

Future<void> _pilihJam() async {
  final hasil = await showTimePicker(
    context: context,
    initialTime: _jam ?? const TimeOfDay(hour: 9, minute: 0),
  );
```

```
  if (hasil != null) setState(() => _jam = hasil);
}
```

`TimeOfDay` hanya berisi jam dan menit (tanpa tanggal). `DateTime` berisi tanggal lengkap. `firstDate` dan `lastDate` membatasi tanggal yang bisa dipilih.

Contoh lengkap: form pengajuan jadwal bimbingan

Trik yang sering dipakai: tanggal dan jam ditampilkan di `TextFormField` dengan `readOnly: true`. Field tidak bisa diketik, tetapi tetap bisa memakai `validator`, dan `onTap` membuka picker.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const JadwalBimbinganScreen(),
    );
  }
}

class JadwalBimbinganScreen extends StatefulWidget {
  const JadwalBimbinganScreen({super.key});

  @override
  State<JadwalBimbinganScreen> createState() => _JadwalBimbinganScreenState();
}

class _JadwalBimbinganScreenState extends State<JadwalBimbinganScreen> {
  final _formKey = GlobalKey<FormState>();
  final _tanggalController = TextEditingController();
  final _jamController = TextEditingController();

  final List<String> _daftarDosen = [
    'Dr. Bambang Susilo, M.Kom.',
    'Nadia Fitri, M.T.',
    'Irham Akbar, M.Cs.',
  ];
  String? _dosen;
  DateTime? _tanggal;
  TimeOfDay? _jam;

  static const _namaBulan = [
    'Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun',
    'Jul', 'Agu', 'Sep', 'Okt', 'Nov', 'Des',
```

```
  ];

  String _formatTanggal(DateTime t) => '${t.day} ${_namaBulan[t.month - 1]} ${t.year}';

  String _formatJam(TimeOfDay j) =>
      '${j.hour.toString().padLeft(2, '0')}:${j.minute.toString().padLeft(2, '0')}';

  @override
  void dispose() {
    _tanggalController.dispose();
    _jamController.dispose();
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final sekarang = DateTime.now();
    final hasil = await showDatePicker(
      context: context,
      initialDate: _tanggal ?? sekarang,
      firstDate: sekarang, // tidak boleh memilih tanggal yang sudah lewat
      lastDate: sekarang.add(const Duration(days: 60)),
      // Hari Sabtu dan Minggu tidak bisa dipilih
      selectableDayPredicate: (hari) =>
          hari.weekday != DateTime.saturday && hari.weekday != DateTime.sunday,
    );
    if (hasil == null) return;
    setState(() {
      _tanggal = hasil;
      _tanggalController.text = _formatTanggal(hasil);
    });
  }

  Future<void> _pilihJam() async {
    final hasil = await showTimePicker(
      context: context,
      initialTime: _jam ?? const TimeOfDay(hour: 9, minute: 0),
    );
    if (hasil == null) return;
    setState(() {
      _jam = hasil;
      _jamController.text = _formatJam(hasil);
    });
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          'Bimbingan dengan $_dosen pada ${_formatTanggal(_tanggal!)} '
          'pukul ${_formatJam(_jam!)}',
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
      appBar: AppBar(title: const Text('Jadwal Bimbingan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            DropdownButtonFormField<String>(
              initialValue: _dosen,
              isExpanded: true,
              decoration: const InputDecoration(
                labelText: 'Dosen Pembimbing',
                prefixIcon: Icon(Icons.person_search),
                border: OutlineInputBorder(),
              ),
              items: _daftarDosen
                  .map((d) => DropdownMenuItem(value: d, child: Text(d)))
                  .toList(),
              onChanged: (baru) => setState(() => _dosen = baru),
              validator: (v) => v == null ? 'Pilih dosen pembimbing' : null,
            ),
            const SizedBox(height: 16),

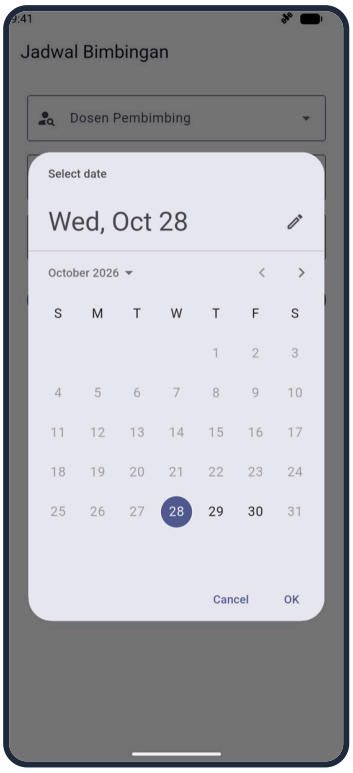
            // TextFormField readOnly: tidak bisa diketik, tapi bisa divalidasi
            TextFormField(
              controller: _tanggalController,
              readOnly: true,
              onTap: _pilihTanggal,
              decoration: const InputDecoration(
                labelText: 'Tanggal',
                prefixIcon: Icon(Icons.calendar_month),
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.isEmpty) ? 'Pilih tanggal' : null,
            ),
            const SizedBox(height: 16),

            TextFormField(
              controller: _jamController,
              readOnly: true,
              onTap: _pilihJam,
              decoration: const InputDecoration(
                labelText: 'Jam',
                prefixIcon: Icon(Icons.access_time),
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (_jam == null) return 'Pilih jam';
                if (_jam!.hour < 8 || _jam!.hour >= 16) {
                  return 'Bimbingan hanya pukul 08:00 - 16:00';
                }
                return null;
              },
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.event_available),
              label: const Text('Ajukan Jadwal'),
            ),
```

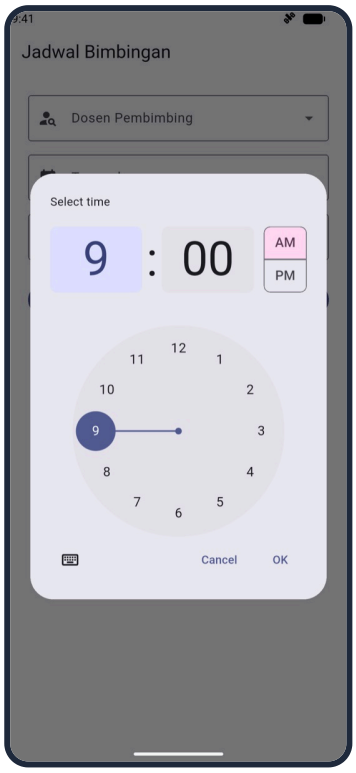
```
            ],
          ),
        ),
      );
    }
  }
```



Ajukan Jadwal ditekan saat form kosong



Dialog DatePicker terbuka saat field Tanggal diketuk



Dialog TimePicker terbuka saat field Jam diketuk

Penjelasan kode:

- `firstDate: sekarang` membuat tanggal yang sudah lewat tidak bisa dipilih. `lastDate` dibatasi 60 hari ke depan.
- `selectableDayPredicate` menerima fungsi yang mengembalikan `false` untuk hari yang tidak boleh dipilih (Sabtu dan Minggu).
- Setelah tanggal dipilih, teksnya ditulis ke `_tanggalController` sehingga tampil di field dan lolos validator.
- Validator jam memeriksa jam kerja (08:00-16:00). Validator tidak harus membaca `v`, bisa juga membaca variabel State seperti `_jam`.
- `_formatTanggal` dan `_formatJam` memformat secara manual. Untuk format yang lebih lengkap (nama hari, bahasa lain), kelak bisa memakai package `intl`.

⚠️ **Kesalahan umum**

- **Error "There should be exactly one item with value ..."** pada dropdown. Terjadi jika nilai terpilih tidak ada di `items` atau ada dua item dengan `value` sama.
- **Teks dropdown panjang overflow.** Tambahkan `isExpanded: true`.
- `initialDate` **di luar rentang** `firstDate` **-** `lastDate`**.** Menyebabkan error. Pastikan nilai awal berada dalam rentang.
- **Tidak memeriksa** `null`**.** Hasil picker bernilai null jika pengguna membatalkan.

9. Umpan Balik: SnackBar, AlertDialog & BottomSheet

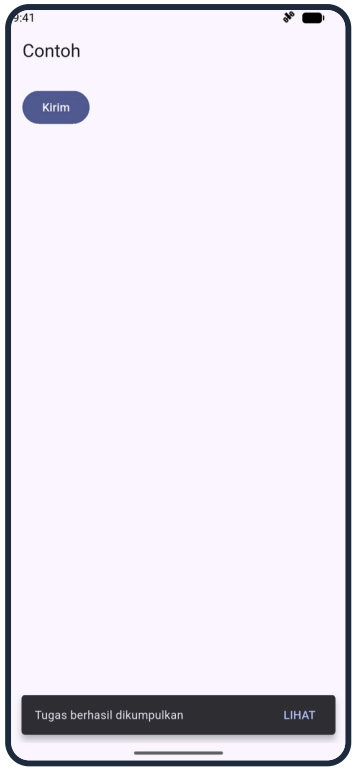
Setelah pengguna menekan tombol, aplikasi harus memberi tanggapan: "berhasil disimpan", "yakin ingin menghapus?", atau menampilkan pilihan tambahan. Tiga alat utamanya adalah **SnackBar**, **AlertDialog**, dan **BottomSheet**.

Widget	Sifat	Kapan dipakai
`SnackBar`	Pesan singkat di bawah layar, hilang sendiri, tidak menghalangi	Konfirmasi berhasil, tombol Urungkan
`AlertDialog`	Kotak di tengah layar, menghentikan interaksi sampai dijawab	Konfirmasi hapus, peringatan penting
`showModalBottomSheet`	Panel dari bawah layar	Daftar pilihan aksi, form singkat

1. SnackBar

SnackBar ditampilkan lewat `ScaffoldMessenger.of(context).showSnackBar(...)`. Tanda `..` (*cascade*) memanggil dua method berturut-turut pada objek yang sama.

```
void _tampilkanSnackBar(BuildContext context) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar() // tutup SnackBar lama agar tidak antre
    ..showSnackBar(
      SnackBar(
        content: const Text('Tugas berhasil dikumpulkan'),
        behavior: SnackBarBehavior.floating, // melayang, sudut membulat
        duration: const Duration(seconds: 3),
        action: SnackBarAction(
          label: 'LIHAT',
          onPressed: () => debugPrint('Buka detail tugas'),
        ),
      ),
    );
}
```



SnackBar mengambang dengan tombol LIHAT

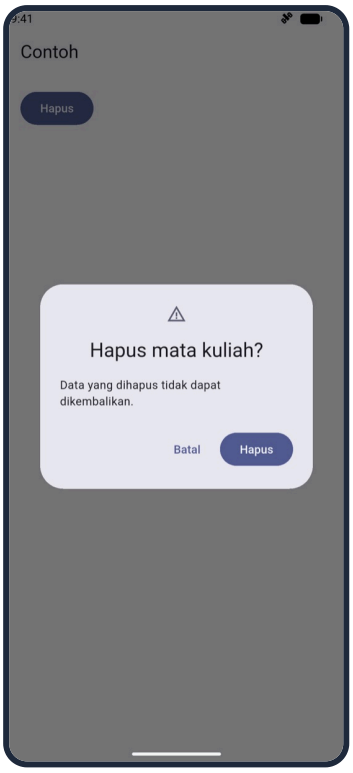
2. AlertDialog yang mengembalikan jawaban

`showDialog<bool>` menunggu sampai dialog ditutup. Nilai yang dikirim lewat `Navigator.pop(dialogContext, nilai)` menjadi hasil `await`.

```
Future<void> _konfirmasiHapus(BuildContext context) async {
  // showDialog mengembalikan nilai yang dikirim lewat Navigator.pop
  final yakin = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      icon: const Icon(Icons.warning_amber),
      title: const Text('Hapus mata kuliah?'),
      content: const Text('Data yang dihapus tidak dapat dikembalikan.'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(dialogContext, true),
          child: const Text('Hapus'),
        ),
      ],
    ),
  );

  // yakin bisa null jika dialog ditutup dengan mengetuk area luar
  if (yakin == true && context.mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Mata kuliah dihapus')),
    );
```

```
  }
}
```



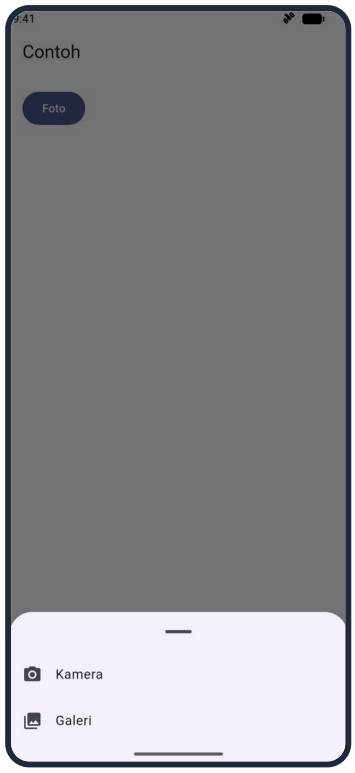
AlertDialog konfirmasi hapus

Perhatikan `context.mounted`: setelah `await`, layar bisa saja sudah ditutup. Memakai `context` yang sudah tidak terpasang akan menimbulkan error, jadi selalu periksa dulu.

3. Modal BottomSheet

```dart
Future<void> _pilihSumberFoto(BuildContext context) async {
  final pilihan = await showModalBottomSheet<String>(
    context: context,
    showDragHandle: true, // garis kecil di atas sheet
    builder: (sheetContext) => SafeArea(
      child: Column(
        mainAxisSize: MainAxisSize.min, // tinggi sheet secukupnya
        children: [
          ListTile(
            leading: const Icon(Icons.camera_alt),
            title: const Text('Kamera'),
            onTap: () => Navigator.pop(sheetContext, 'kamera'),
          ),
          ListTile(
            leading: const Icon(Icons.photo_library),
            title: const Text('Galeri'),
            onTap: () => Navigator.pop(sheetContext, 'galeri'),
          ),
        ],
      ),
    ),
  );
```

```
  debugPrint('Dipilih: $pilihan'); // null jika sheet ditutup tanpa memilih
}
```



BottomSheet berisi pilihan Kamera dan Galeri

`mainAxisSize: MainAxisSize.min` membuat tinggi sheet mengikuti isinya. `SafeArea` mencegah isi tertutup bilah navigasi HP.

Contoh lengkap: catatan tugas dengan tiga jenis umpan balik

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const CatatanTugasScreen(),
    );
  }
}

class CatatanTugasScreen extends StatefulWidget {
```

```dart
  const CatatanTugasScreen({super.key});

  @override
  State<CatatanTugasScreen> createState() => _CatatanTugasScreenState();
}

class _CatatanTugasScreenState extends State<CatatanTugasScreen> {
  final List<String> _tugas = [
    'Laporan Praktikum Basis Data',
    'Kuis Pemrograman Mobile',
    'Makalah Etika Profesi',
  ];

  // ---------- 1. SnackBar dengan tombol UNDO ----------
  void _hapusDenganUndo(int index) {
    final terhapus = _tugas[index];
    setState(() => _tugas.removeAt(index));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('"$terhapus" dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            onPressed: () => setState(() => _tugas.insert(index, terhapus)),
          ),
        ),
      );
  }

  // ---------- 2. AlertDialog konfirmasi ----------
  Future<void> _konfirmasiHapusSemua() async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus semua tugas?'),
        content: Text('${_tugas.length} tugas akan dihapus.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus Semua'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    setState(() => _tugas.clear());
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Semua tugas dihapus')),
    );
  }

  // ---------- 3. Dialog berisi TextField untuk menambah data ----------
  Future<void> _dialogTambah() async {
```

```dart
    // Cukup simpan teks di variabel lokal lewat onChanged.
    // (Membuat TextEditingController di sini berisiko: controller harus
    // di-dispose, padahal TextField masih hidup selama animasi dialog ditutup.)
    var teks = '';
    final judul = await showDialog<String>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Tugas baru'),
        content: TextField(
          autofocus: true,
          decoration: const InputDecoration(hintText: 'Judul tugas'),
          onChanged: (v) => teks = v,
          onSubmitted: (v) => Navigator.pop(dialogContext, v),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, teks),
            child: const Text('Tambah'),
          ),
        ],
      ),
    );

    if (judul == null || judul.trim().isEmpty || !mounted) return;
    setState(() => _tugas.add(judul.trim()));
  }

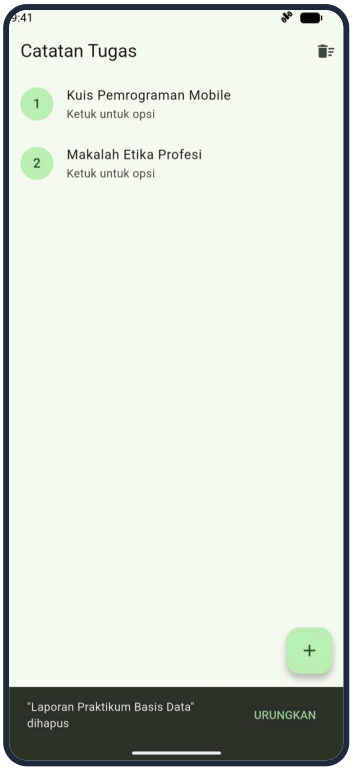
  // ---------- 4. BottomSheet berisi pilihan aksi ----------
  Future<void> _menuTugas(int index) async {
    final aksi = await showModalBottomSheet<String>(
      context: context,
      showDragHandle: true,
      builder: (sheetContext) => SafeArea(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Padding(
              padding: const EdgeInsets.all(8),
              child: Text(_tugas[index], style: const TextStyle(fontWeight: FontWeight.bold)),
            ),
            ListTile(
              leading: const Icon(Icons.check_circle_outline),
              title: const Text('Tandai selesai'),
              onTap: () => Navigator.pop(sheetContext, 'selesai'),
            ),
            ListTile(
              leading: const Icon(Icons.delete_outline),
              title: const Text('Hapus'),
              onTap: () => Navigator.pop(sheetContext, 'hapus'),
            ),
          ],
        ),
      ),
    );
    if (!mounted) return;
```

```
    if (aksi == 'hapus') _hapusDenganUndo(index);
    if (aksi == 'selesai') {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Mantap! "${_tugas[index]}" selesai'),
          behavior: SnackBarBehavior.floating,
        ),
      );
    }
  }

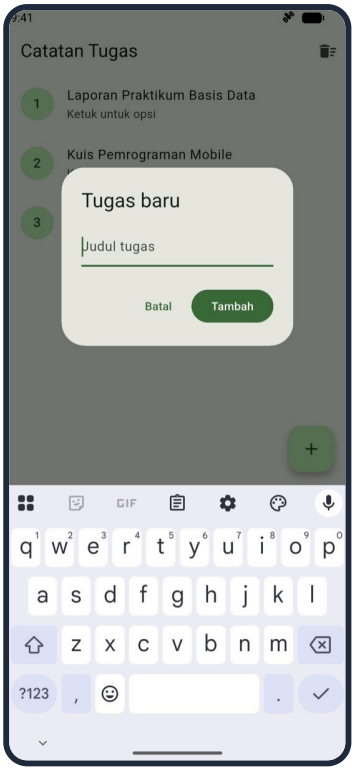
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Tugas'),
        actions: [
          IconButton(
            tooltip: 'Hapus semua',
            icon: const Icon(Icons.delete_sweep),
            onPressed: _tugas.isEmpty ? null : _konfirmasiHapusSemua,
          ),
        ],
      ),
      body: _tugas.isEmpty
          ? const Center(child: Text('Tidak ada tugas. Santai dulu!'))
          : ListView(
              children: [
                for (var i = 0; i < _tugas.length; i++)
                  ListTile(
                    leading: CircleAvatar(child: Text('${i + 1}')),
                    title: Text(_tugas[i]),
                    subtitle: const Text('Ketuk untuk opsi'),
                    onTap: () => _menuTugas(i),
                  ),
              ],
            ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah tugas',
        onPressed: _dialogTambah,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Menu bottom sheet saat tugas diketuk



SnackBar dengan tombol URUNGKAN setelah tugas dihapus



Dialog berisi TextField untuk menambah tugas

Penjelasan kode:

- **Urungkan:** sebelum menghapus, item dan posisinya disimpan (`terhapus` , `index`). Tombol URUNGKAN memasukkannya kembali dengan `insert(index, terhapus)` .
- **Hapus semua:** dialog mengembalikan `bool?` . Ditulis `yakin != true` agar nilai `null` (dialog ditutup dengan mengetuk luar) dianggap batal.
- **Tambah:** dialog bisa berisi TextField. Teks disimpan ke variabel lokal lewat `onChanged` lalu dikirim lewat `Navigator.pop` .
- **Menu tugas:** bottom sheet mengembalikan `String` berisi aksi yang dipilih, lalu diproses setelah sheet tertutup.
- Tombol hapus semua di AppBar diberi `onPressed: _tugas.isEmpty ? null : ...` sehingga nonaktif saat daftar kosong.

⚠️ **Kesalahan umum**

- **Memakai `context` milik layar untuk menutup dialog.** Gunakan context dari `builder` dialog (`dialogContext`) di `Navigator.pop`.
- **SnackBar menumpuk dan antre lama.** Panggil `hideCurrentSnackBar()` sebelum menampilkan yang baru.
- **Lupa memeriksa `mounted` setelah `await showDialog`.** Muncul peringatan "use_build_context_synchronously".
- **BottomSheet berisi form tertutup keyboard.** Gunakan `isScrollControlled: true` dan padding sebesar `MediaQuery.viewInsetsOf(context).bottom` (lihat studi kasus Modul 06).

10. Studi Kasus: Formulir Pendaftaran Mahasiswa

Sekarang kita gabungkan semua materi modul ini menjadi satu aplikasi utuh: **formulir pendaftaran mahasiswa baru** yang memakai TextFormField, validasi, password, DatePicker, Radio, Dropdown, Slider, FilterChip, Switch, Checkbox, dialog konfirmasi, dan halaman hasil.

Rancangan aplikasi

1. **Model data** `DataPendaftar`: satu objek yang menampung semua isian. Lebih rapi daripada mengirim sepuluh variabel terpisah.
2. **Validators**: aturan NIM, email, nomor HP, password, dan konfirmasi.
3. **PendaftaranScreen**: form tiga bagian (Data Diri, Kontak & Akun, Data Akademik).
4. **HasilPendaftaranScreen**: menampilkan ringkasan data. Perpindahan halaman memakai `Navigator.push`, yang akan dibahas tuntas di Modul 07.

Alur saat tombol Kirim ditekan

1. Tutup keyboard dengan `unfocus()`.
2. Jalankan `validate()` untuk semua TextFormField dan Dropdown.
3. Periksa manual input yang bukan FormField: jenis kelamin, minat, persetujuan.
4. Kumpulkan isian ke objek `DataPendaftar`.
5. Tampilkan dialog konfirmasi. Jika disetujui, buka halaman hasil.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pendaftaran Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PendaftaranScreen(),
    );
  }
}

// ======================== MODEL DATA ========================
enum JenisKelamin { lakiLaki, perempuan }

class DataPendaftar {
```

```dart
  final String nama;
  final String nim;
  final String email;
  final String noHp;
  final String prodi;
  final JenisKelamin jenisKelamin;
  final DateTime tanggalLahir;
  final List<String> minat;
  final int semester;
  final bool ikutBeasiswa;

  const DataPendaftar({
    required this.nama,
    required this.nim,
    required this.email,
    required this.noHp,
    required this.prodi,
    required this.jenisKelamin,
    required this.tanggalLahir,
    required this.minat,
    required this.semester,
    required this.ikutBeasiswa,
  });
}

// ======================= VALIDATOR =======================
class Validators {
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? v, String field) =>
      (v == null || v.trim().isEmpty) ? '$field wajib diisi' : null;

  static String? nim(String? v) {
    if (v == null || v.trim().isEmpty) return 'NIM wajib diisi';
    if (!RegExp(r'^[A-Z]\d[A-Z]\d{6}$').hasMatch(v.trim().toUpperCase())) {
      return 'Format NIM contoh: H1D024001';
    }
    return null;
  }

  static String? email(String? v) {
    if (v == null || v.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(v.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? noHp(String? v) {
    if (v == null || v.trim().isEmpty) return 'Nomor HP wajib diisi';
    if (!RegExp(r'^08\d{8,11}$').hasMatch(v.trim())) {
      return 'Nomor HP diawali 08, 10-13 digit';
    }
    return null;
  }

  static String? password(String? v) {
    if (v == null || v.isEmpty) return 'Password wajib diisi';
    if (v.length < 8) return 'Password minimal 8 karakter';
    return null;
  }
```

```dart
  static String? konfirmasi(String? v, String asli) {
    if (v == null || v.isEmpty) return 'Ulangi password';
    if (v != asli) return 'Konfirmasi password tidak sama';
    return null;
  }
}

String formatTanggal(DateTime t) {
  const bulan = [
    'Januari', 'Februari', 'Maret', 'April', 'Mei', 'Juni',
    'Juli', 'Agustus', 'September', 'Oktober', 'November', 'Desember',
  ];
  return '${t.day} ${bulan[t.month - 1]} ${t.year}';
}

// ======================== LAYAR FORM ========================
class PendaftaranScreen extends StatefulWidget {
  const PendaftaranScreen({super.key});

  @override
  State<PendaftaranScreen> createState() => _PendaftaranScreenState();
}

class _PendaftaranScreenState extends State<PendaftaranScreen> {
  final _formKey = GlobalKey<FormState>();

  final _namaC = TextEditingController();
  final _nimC = TextEditingController();
  final _emailC = TextEditingController();
  final _hpC = TextEditingController();
  final _passwordC = TextEditingController();
  final _konfirmasiC = TextEditingController();
  final _tanggalC = TextEditingController();

  static const _daftarProdi = [
    'S1 Informatika',
    'S1 Teknik Elektro',
    'S1 Teknik Sipil',
    'S1 Teknik Industri',
  ];
  static const _daftarMinat = ['Mobile', 'Web', 'AI', 'Jaringan', 'UI/UX'];

  String? _prodi;
  JenisKelamin? _jenisKelamin;
  DateTime? _tanggalLahir;
  final Set<String> _minat = {};
  double _semester = 1;
  bool _ikutBeasiswa = false;
  bool _setuju = false;
  bool _sembunyikan = true;

  @override
  void dispose() {
    for (final c in [_namaC, _nimC, _emailC, _hpC, _passwordC, _konfirmasiC, _tanggalC]) {
      c.dispose();
    }
    super.dispose();
  }
```

```dart
Future<void> _pilihTanggalLahir() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggalLahir ?? DateTime(2005, 1, 1),
    firstDate: DateTime(1990),
    lastDate: DateTime.now(),
    helpText: 'Tanggal lahir',
  );
  if (hasil == null) return;
  setState(() {
    _tanggalLahir = hasil;
    _tanggalC.text = formatTanggal(hasil);
  });
}

void _tampilkanError(String pesan) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(pesan), backgroundColor: Colors.red));
}

Future<void> _kirim() async {
  FocusScope.of(context).unfocus();

  // 1. Validasi semua TextFormField & DropdownButtonFormField
  final formValid = _formKey.currentState!.validate();
  if (!formValid) {
    _tampilkanError('Masih ada isian yang salah');
    return;
  }
  // 2. Validasi input yang tidak memakai FormField
  if (_jenisKelamin == null) {
    _tampilkanError('Pilih jenis kelamin');
    return;
  }
  if (_minat.isEmpty) {
    _tampilkanError('Pilih minimal satu bidang minat');
    return;
  }
  if (!_setuju) {
    _tampilkanError('Centang pernyataan persetujuan');
    return;
  }

  // 3. Kumpulkan data ke satu objek
  final data = DataPendaftar(
    nama: _namaC.text.trim(),
    nim: _nimC.text.trim().toUpperCase(),
    email: _emailC.text.trim(),
    noHp: _hpC.text.trim(),
    prodi: _prodi!,
    jenisKelamin: _jenisKelamin!,
    tanggalLahir: _tanggalLahir!,
    minat: _minat.toList(),
    semester: _semester.round(),
    ikutBeasiswa: _ikutBeasiswa,
  );
```

```
    // 4. Konfirmasi lewat dialog
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Kirim pendaftaran?'),
        content: Text('Pastikan data ${data.nama} (${data.nim}) sudah benar.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Periksa lagi'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Ya, kirim'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;

    // 5. Tampilkan hasil di layar baru (detail navigasi dibahas di Modul 07)
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => HasilPendaftaranScreen(data: data)),
    );
  }

  Widget _judulBagian(String teks) => Padding(
        padding: const EdgeInsets.only(top: 24, bottom: 8),
        child: Text(teks, style: Theme.of(context).textTheme.titleMedium),
      );

  @override
  Widget build(BuildContext context) {
    const jarak = SizedBox(height: 12);

    return Scaffold(
      appBar: AppBar(title: const Text('Pendaftaran Mahasiswa Baru')),
      body: Form(
        key: _formKey,
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.fromLTRB(16, 0, 16, 32),
          children: [
            _judulBagian('A. Data Diri'),
            TextFormField(
              controller: _namaC,
              textCapitalization: TextCapitalization.words,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person_outline),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.wajib(v, 'Nama'),
            ),
            jarak,
            TextFormField(
              controller: _nimC,
```

```
            textCapitalization: TextCapitalization.characters,
            textInputAction: TextInputAction.next,
            decoration: const InputDecoration(
              labelText: 'NIM',
              hintText: 'H1D024001',
              prefixIcon: Icon(Icons.badge_outlined),
              border: OutlineInputBorder(),
            ),
            validator: Validators.nim,
          ),
          jarak,
          TextFormField(
            controller: _tanggalC,
            readOnly: true,
            onTap: _pilihTanggalLahir,
            decoration: const InputDecoration(
              labelText: 'Tanggal Lahir',
              prefixIcon: Icon(Icons.cake_outlined),
              border: OutlineInputBorder(),
            ),
            validator: (v) => Validators.wajib(v, 'Tanggal lahir'),
          ),
          jarak,
          const Text('Jenis Kelamin'),
          RadioGroup<JenisKelamin>(
            groupValue: _jenisKelamin,
            onChanged: (v) => setState(() => _jenisKelamin = v),
            child: const Row(
              children: [
                Expanded(
                  child: RadioListTile(
                    value: JenisKelamin.lakiLaki,
                    title: Text('Laki-laki'),
                    contentPadding: EdgeInsets.zero,
                  ),
                ),
                Expanded(
                  child: RadioListTile(
                    value: JenisKelamin.perempuan,
                    title: Text('Perempuan'),
                    contentPadding: EdgeInsets.zero,
                  ),
                ),
              ],
            ),
          ),

          _judulBagian('B. Kontak & Akun'),
          TextFormField(
            controller: _emailC,
            keyboardType: TextInputType.emailAddress,
            textInputAction: TextInputAction.next,
            decoration: const InputDecoration(
              labelText: 'Email',
              prefixIcon: Icon(Icons.email_outlined),
              border: OutlineInputBorder(),
            ),
            validator: Validators.email,
          ),
```

```
            jarak,
            TextFormField(
              controller: _hpC,
              keyboardType: TextInputType.phone,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Nomor HP',
                hintText: '081234567890',
                prefixIcon: Icon(Icons.phone_outlined),
                border: OutlineInputBorder(),
              ),
              validator: Validators.noHp,
            ),
            jarak,
            TextFormField(
              controller: _passwordC,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.next,
              decoration: InputDecoration(
                labelText: 'Password',
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
                  onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
                ),
              ),
              validator: Validators.password,
            ),
            jarak,
            TextFormField(
              controller: _konfirmasiC,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.done,
              decoration: const InputDecoration(
                labelText: 'Konfirmasi Password',
                prefixIcon: Icon(Icons.lock_reset),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.konfirmasi(v, _passwordC.text),
            ),

            _judulBagian('C. Data Akademik'),
            DropdownButtonFormField<String>(
              initialValue: _prodi,
              isExpanded: true,
              decoration: const InputDecoration(
                labelText: 'Program Studi',
                prefixIcon: Icon(Icons.school_outlined),
                border: OutlineInputBorder(),
              ),
              items: _daftarProdi
                  .map((p) => DropdownMenuItem(value: p, child: Text(p)))
                  .toList(),
              onChanged: (v) => setState(() => _prodi = v),
              validator: (v) => v == null ? 'Pilih program studi' : null,
            ),
            jarak,
            Text('Semester saat ini: ${_semester.round()}'),
```

```
            Slider(
              value: _semester,
              min: 1,
              max: 14,
              divisions: 13,
              label: 'Semester ${_semester.round()}',
              onChanged: (v) => setState(() => _semester = v),
            ),
            const Text('Bidang minat (boleh lebih dari satu)'),
            const SizedBox(height: 8),
            Wrap(
              spacing: 8,
              children: [
                for (final m in _daftarMinat)
                  // FilterChip = versi "chip" dari checkbox
                  FilterChip(
                    label: Text(m),
                    selected: _minat.contains(m),
                    onSelected: (pilih) => setState(() {
                      pilih ? _minat.add(m) : _minat.remove(m);
                    }),
                  ),
              ],
            ),
            SwitchListTile(
              contentPadding: EdgeInsets.zero,
              title: const Text('Mengajukan beasiswa'),
              value: _ikutBeasiswa,
              onChanged: (v) => setState(() => _ikutBeasiswa = v),
            ),
            CheckboxListTile(
              contentPadding: EdgeInsets.zero,
              controlAffinity: ListTileControlAffinity.leading,
              title: const Text('Saya menyatakan data di atas benar.'),
              value: _setuju,
              onChanged: (v) => setState(() => _setuju = v ?? false),
            ),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: _kirim,
              icon: const Icon(Icons.send),
              label: const Text('Kirim Pendaftaran'),
              style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(50)),
            ),
          ],
        ),
      ),
    );
  }
}

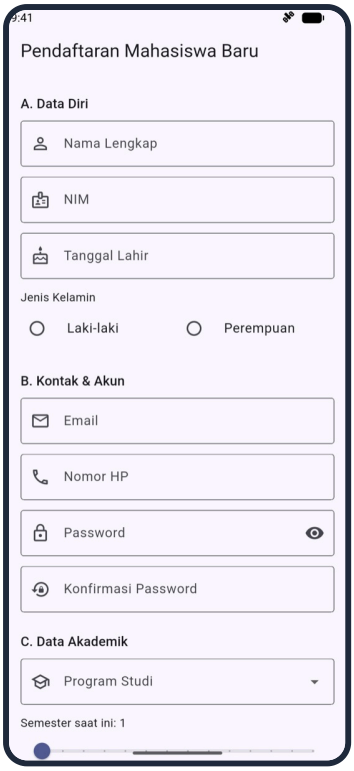
// ======================== LAYAR HASIL ========================
class HasilPendaftaranScreen extends StatelessWidget {
  final DataPendaftar data;

  const HasilPendaftaranScreen({super.key, required this.data});

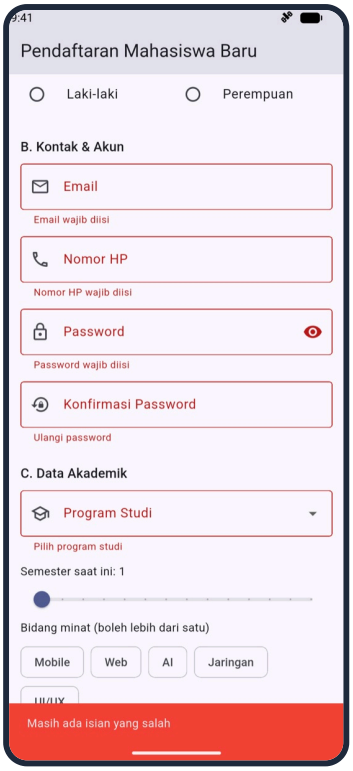
  @override
  Widget build(BuildContext context) {
```

```
    final baris = <(IconData, String, String)>[
      (Icons.person, 'Nama', data.nama),
      (Icons.badge, 'NIM', data.nim),
      (Icons.wc, 'Jenis Kelamin',
          data.jenisKelamin == JenisKelamin.lakiLaki ? 'Laki-laki' : 'Perempuan'),
      (Icons.cake, 'Tanggal Lahir', formatTanggal(data.tanggalLahir)),
      (Icons.email, 'Email', data.email),
      (Icons.phone, 'Nomor HP', data.noHp),
      (Icons.school, 'Program Studi', data.prodi),
      (Icons.timeline, 'Semester', '${data.semester}'),
      (Icons.interests, 'Minat', data.minat.join(', ')),
      (Icons.card_giftcard, 'Beasiswa', data.ikutBeasiswa ? 'Ya' : 'Tidak'),
    ];

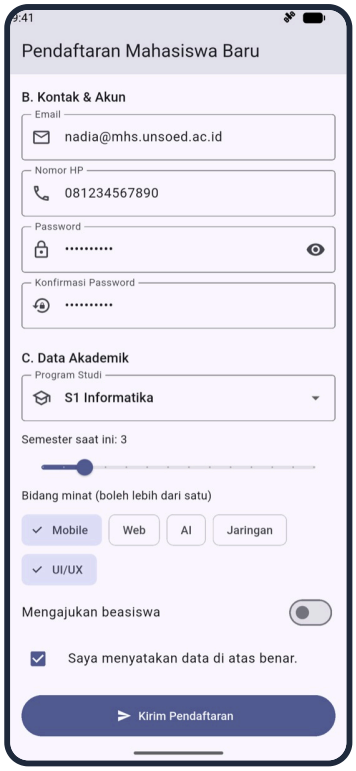
    return Scaffold(
      appBar: AppBar(title: const Text('Pendaftaran Berhasil')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Icon(Icons.check_circle, color: Colors.green, size: 80),
          const SizedBox(height: 8),
          Text(
            'Selamat datang, ${data.nama}!',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.titleLarge,
          ),
          const SizedBox(height: 16),
          Card(
            child: Column(
              children: [
                for (final (ikon, label, nilai) in baris)
                  ListTile(
                    leading: Icon(ikon),
                    title: Text(label),
                    subtitle: Text(nilai),
                  ),
              ],
            ),
          ),
          const SizedBox(height: 16),
          OutlinedButton.icon(
            onPressed: () => Navigator.pop(context),
            icon: const Icon(Icons.edit),
            label: const Text('Kembali & ubah data'),
          ),
        ],
      ),
    );
  }
}
```



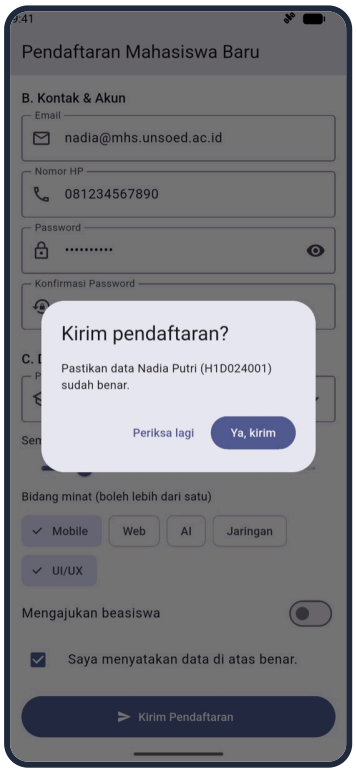
Bagian atas form pendaftaran



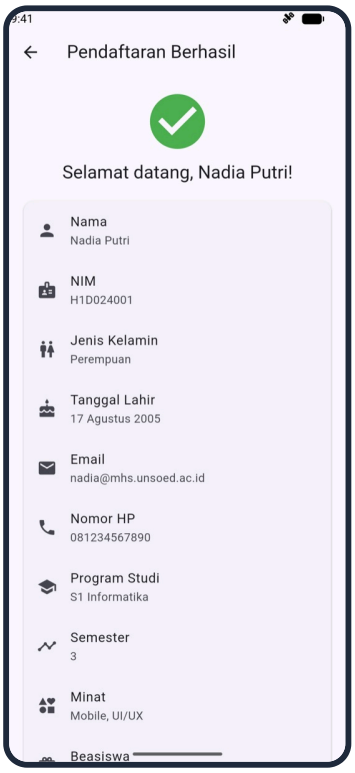
Kirim ditekan saat form kosong: pesan error dan SnackBar merah



Bagian Data Akademik: Dropdown, Slider, FilterChip, Switch, Checkbox



Dialog konfirmasi setelah semua isian valid



Halaman hasil pendaftaran

Penjelasan bagian penting:

- **Validasi NIM** memakai pola `^[A-Z]\d[A-Z]\d{6}$` : huruf, angka, huruf, lalu enam angka (contoh H1D024001). Isian diubah ke huruf besar dulu dengan `toUpperCase()`.

- **Banyak controller** di-dispose sekaligus dengan perulangan `for (final c in [...]) c.dispose();`.
- **FilterChip** berfungsi seperti checkbox dalam bentuk chip. Pilihan disimpan di `Set<String>` agar tidak ada duplikat.
- **Radio dalam Row**: dua `RadioListTile` dibungkus `Expanded` agar berbagi lebar, dan `RadioGroup` membungkus Row tersebut.
- **Record** `(IconData, String, String)` di halaman hasil adalah fitur Dart 3 untuk mengelompokkan beberapa nilai tanpa membuat kelas. `for (final (ikon, label, nilai) in baris)` langsung memecahnya menjadi tiga variabel.
- **Data dikirim ke halaman hasil** lewat konstruktor: `HasilPendaftaranScreen(data: data)`.

> 💡 **Uji form Anda seperti penguji**
>
> - Tekan Kirim tanpa mengisi apa pun: semua pesan error harus muncul.
> - Isi NIM dengan huruf kecil (h1d024001): harus tetap diterima.
> - Isi email `budi@`, nomor HP `0812`, password 5 huruf: masing-masing harus ditolak dengan pesan yang jelas.
> - Isi password dan konfirmasi berbeda satu huruf: harus ditolak.
> - Isi semua dengan benar tetapi tidak memilih minat: SnackBar harus muncul.

> ℹ️ **Dipakai lagi di Modul 13-14**
>
> Kelas `Validators`, pola `GlobalKey<FormState>`, toggle password, dan tombol yang nonaktif saat mengirim akan dipakai ulang untuk layar login dan register pada proyek CRUD + autentikasi. Simpan kode ini baik-baik.

Tugas Praktikum: Praktikum 5: Formulir Pendaftaran Seminar Tugas Akhir

Sasaran Praktikum

Membangun formulir lengkap dengan validasi, input pilihan, dan umpan balik untuk pendaftaran seminar tugas akhir mahasiswa.

Langkah Pengerjaan

1. Buat proyek baru dan file `lib/main.dart` berisi `FormSeminarScreen` (StatefulWidget).
2. Buat kelas `Validators` berisi fungsi static untuk: wajib diisi, NIM (format seperti H1D024001), email, dan panjang minimal.
3. Tambahkan field: NIM, Nama, Email, Judul Tugas Akhir (multi-baris, minimal 20 karakter), dan Ringkasan (multi-baris, maksimal 300 karakter dengan `maxLength`).
4. Atur `keyboardType`, `textInputAction: TextInputAction.next`, dan `FocusNode` agar tombol Next berpindah ke field berikutnya.
5. Tambahkan `DropdownButtonFormField` untuk Dosen Pembimbing 1 dan 2 dengan validator, serta tolak jika keduanya sama.
6. Tambahkan pemilih tanggal seminar dengan `showDatePicker` (hanya hari kerja, mulai besok) dan jam dengan `showTimePicker`, ditampilkan pada TextFormField `readOnly`.
7. Tambahkan `RadioGroup` untuk jenis seminar (Proposal / Hasil / Sidang) dan `CheckboxListTile` persetujuan.
8. Saat Kirim ditekan dan semua valid, tampilkan `AlertDialog` ringkasan data; jika dikonfirmasi tampilkan `SnackBar` sukses dan kosongkan form. Tombol Kirim harus nonaktif selama simulasi pengiriman 2 detik.
9. Pastikan semua `TextEditingController` dan `FocusNode` di-dispose, dan `flutter analyze` tidak menampilkan peringatan.

Output yang Diharapkan

Formulir yang menolak isian salah dengan pesan error yang jelas, berpindah fokus dengan tombol Next, menampilkan dialog ringkasan, dan SnackBar sukses setelah dikirim.

Rubrik Penilaian

Form + GlobalKey<FormState> + kelas Validators (30%), Input pilihan: Dropdown, Radio, Checkbox, Date/TimePicker (25%), Keyboard, FocusNode, dan dispose yang benar (20%), Umpan balik dialog/SnackBar dan tombol nonaktif saat mengirim (15%), Kerapian UI (10%).

Kuis Pemahaman

1. Mengapa kita WAJIB memanggil method controller.dispose() di dalam method dispose() milik StatefulWidget?

A. Untuk mencegah memory leak (kebocoran memori) karena listener controller terus hidup meskipun widget sudah dihancurkan

B. Karena jika tidak ditutup, data form akan otomatis terkirim ke server

C. Untuk mengubah font teks menjadi tebal

D. Agar tombol submit bisa diklik

2. Apa nilai kembalian (return value) dari fungsi validator pada TextFormField jika input pengguna SUDAH BENAR dan VALID?

A. Mengembalikan String 'Valid'

B. Mengembalikan nilai boolean true

C. Mengembalikan null

D. Mengembalikan angka 1

3. Bagaimana cara membuat sebuah FilledButton tampil nonaktif (disabled)?

A. Memberi onPressed: null

B. Memberi child: null

C. Membungkusnya dengan Opacity(opacity: 0.5)

D. Memberi enabled: false pada FilledButton

4. Efek riak (ripple) InkWell tidak terlihat sama sekali. Penyebab yang paling mungkin adalah...

A. InkWell tidak memiliki widget Material di atasnya, atau anaknya Container berwarna yang menutupi riak

B. InkWell hanya bekerja di iOS

C. onTap pada InkWell bernilai fungsi

D. InkWell dibungkus Padding

5. Apa perbedaan utama onChanged dan TextEditingController pada TextField?

A. onChanged hanya membaca saat pengguna mengetik, sedangkan controller dapat membaca, mengubah, dan mengosongkan teks dari kode kapan saja

B. onChanged lebih hemat memori sehingga controller tidak perlu dipakai

C. Controller hanya bisa dipakai pada TextFormField

D. Tidak ada perbedaan

6. Kode manakah yang memindahkan kursor ke field password ketika tombol Next pada keyboard ditekan di field email?

A. textInputAction: TextInputAction.next, onSubmitted: (_) ⇒ _passwordFocus.requestFocus()

B. keyboardType: TextInputType.next

C. obscureText: true pada field email

D. autofocus: true pada field email

7. Validator konfirmasi password yang benar adalah...

A. (v) ⇒ v != _passwordController.text ? 'Konfirmasi password tidak sama' : null

B. (v) ⇒ v == _passwordController.text ? 'Valid' : 'Tidak sama'

C. (v) ⇒ v!.length >= 8

D. (v) ⇒ _formKey.currentState!.validate()

8. Sejak Flutter 3.35, bagaimana cara yang tidak deprecated untuk membuat sekelompok radio button?

A. Membungkus semua RadioListTile dengan RadioGroup yang memegang groupValue dan onChanged, dan setiap radio hanya diberi value

B. Memberi groupValue dan onChanged pada setiap Radio

C. Memakai Checkbox dengan tristate: true

D. Memakai Switch untuk setiap pilihan

Kunci Jawaban & Pembahasan

1. A — TextEditingController memiliki listener internal yang didaftarkan ke sistem operasi. Jika tidak di-dispose saat widget dihancurkan, referensi controller tetap menggantung di memori, menyebabkan kebocoran memori (memory leak).

2. C — Dalam Flutter Form validation, jika fungsi validator mengembalikan nilai null, framework menganggap isian tersebut valid. Jika tidak valid, fungsi mengembalikan String pesan kesalahan yang akan dicetak merah di bawah input field.

3. A — Semua tombol Material menjadi nonaktif secara otomatis ketika onPressed bernilai null. Warnanya memudar dan tombol tidak merespons sentuhan.

4. A — InkWell menggambar riaknya pada widget Material terdekat. Tanpa Material, atau jika anak InkWell adalah Container berwarna, riak tidak terlihat. Berikan warna pada Material atau gunakan Card.

5. A — onChanged adalah callback yang menerima teks terbaru saat pengguna mengetik. TextEditingController memberi akses penuh: .text untuk membaca/mengubah, clear() untuk mengosongkan, dan addListener untuk mendengarkan perubahan.

6. A — textInputAction mengubah tombol Enter menjadi Next, dan onSubmitted dipanggil saat tombol itu ditekan. FocusNode.requestFocus() memindahkan fokus ke field yang memakai FocusNode tersebut.

7. A — Validator harus mengembalikan null jika valid dan String pesan error jika tidak valid. Isian konfirmasi dibandingkan dengan teks pada controller password.

8. A — Parameter groupValue dan onChanged pada Radio/RadioListTile sudah deprecated. RadioGroup<T> kini memegang nilai terpilih dan callback untuk semua radio di dalamnya.